M256 Unit 13
UNDERGRADUATE COMPUTING

Software development with Java

Building user interfaces

Unit 13

This publication forms part of an Open University course M256 *Software development with Java.* Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at http://www.open.ac.uk where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit http://www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2007.

Edited and designed by The Open University.

Typeset by The Open University.

Printed and bound in Malta by Gutenberg Press.

ISBN 978 0 7492 1561 3

1.1

CONTENTS

M256 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Sarah Mattingly, Course Chair and Author

Lindsey Court, Author

Marion Edwards, Software Developer

Rob Griffiths, Critical Reader

Benedict Heal, Critical Reader

Simon Holland, Author

Barbara Poniatowska, Course Manager

Barbara Segal, Author

Rita Tingle, Author

Richard Walker, Author and Critical Reader

Robin Walker, Author and Critical Reader

Ray Weedon, Academic Editor

Ray Welland, External Assessor, University of Glasgow

Julia White, Course Manager

Ian Blackham, Editor

Phillip Howe, Compositor

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1 Introduction

The most important part of any software project is the people who will eventually use it. Software must fulfil its users' requirements, and they must find it usable.

In your journey through software development, you began by looking at how to specify software requirements. From these requirements, you developed a conceptual model. This led to an implementation model, and then to the creation and testing of a core system. After that you went on to consider how to design an interface that users can interact with effectively.

This unit completes the process of developing a new system by looking at how to *build*, that is, plan and implement, a user interface, and then perform acceptance testing, that is, test the complete system – core system and user interface – as it will be experienced by users. As such it is a very practical unit: much of your study time will be spent doing exercises on your computer.

Figure 1 shows the place of these phases in the development process as introduced in M256.

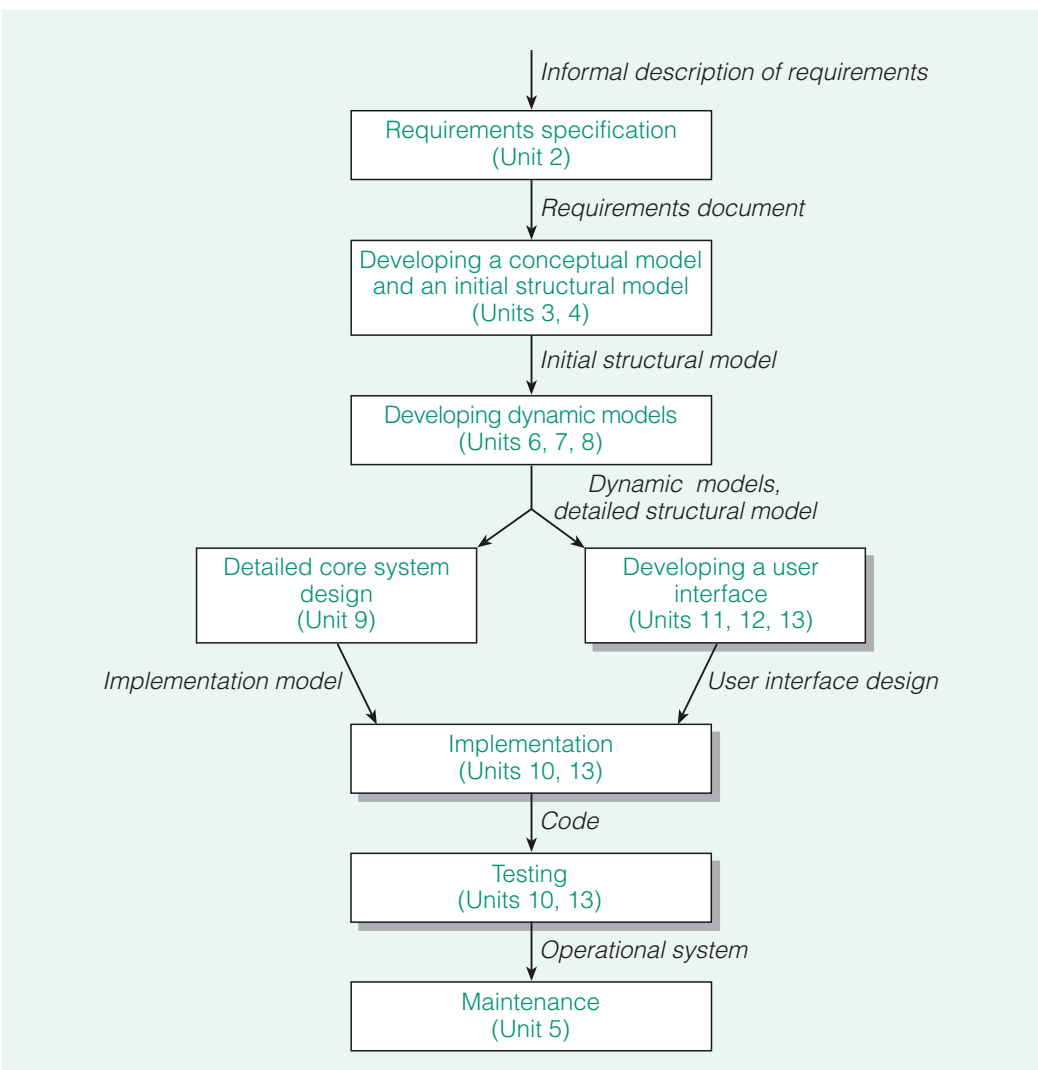


Figure 1 The place of GUI implementation and acceptance testing in the software development process

1.1 The aims of this unit

In this unit you will learn about:

- attaching a user interface to a core system (Section 2);
- implementing graphical user interfaces (GUIs, or GUI interfaces) from reusable components (Sections 2, 3, 4 and 5);
- a systematic approach to building a graphical user interface (Section 3);
- some common issues that arise in building graphical user interfaces (Sections 2 and 3);
- testing a completed system to see whether it meets the users' needs (Section 6).

We emphasise that in M256 you are *not* expected to implement GUIs from scratch. You will only be asked to complete the implementation of GUIs for which most of the code already exists.

1.2 Studying this unit

This unit is mainly practical so you will need access to your computer throughout your study. It contains a number of SAQs and paper-based exercises which are designed to reinforce your understanding of the concepts presented. These form an important part of the teaching strategy and you should work through them as they arise, and read the solutions provided before moving on.

The unit assumes that you have worked through Section 5 of the *NetBeans Guide*, which introduces the NetBeans GUI designer. You may need to refer again to the *NetBeans Guide* during your work on this unit.

It is also assumed that you are familiar with the Java classes whose instances provide GUI components. You do not have to be expert in either of these things – this unit contains detailed instructions for the tasks involved – but you do need to be familiar with the background ideas and the use of NetBeans in general. For reference, the Abstract Windowing Toolkit (AWT) and Swing classes used in this unit are summarised in the Appendix. If you have not used them for some time you may find it useful to refresh your memory by reading through this additional material.

We estimate that Sections 2, 3 and 5 will each take approximately the same amount of study time, and so will Sections 4 and 6, with the latter sections being shorter than the former.

2 Implementing simple user interfaces

In this section you will be introduced to what is involved in implementing a user interface, using a scaled-down version of the Hospital System called the Cottage Hospital System.

The section aims to:

- produce a textual interface to the system for a particular use case;
- produce a GUI interface to the system for the same use case;
- compare the development of a GUI interface with the component-based development that you learnt about in *Unit 5*;
- review the benefits of making a user interface independent of the core system.

To prepare for this we first describe the Cottage Hospital System.

2.1 Introducing the Cottage Hospital System

A cottage hospital is a small hospital providing mainly local health care.

The cottage hospital consists of wards, each of which has patients on it: there are no doctors or teams. Consequently, all that the Cottage Hospital System can do is list the patients on a ward or add a new patient to a ward (for simplicity, patients may not be removed from a ward!). As you might expect, as well as a coordinating class, `CottageHospCoord`, the Cottage Hospital core system defines classes `Ward` and `Patient`.

Each `Ward` object has a `name` instance variable which references a `String` object; each `Patient` object has a `name` instance variable, which also references a `String` object, and a `dateOfBirth` instance variable which references an `M256Date` object.

The system has two use cases, *List Cottage Ward's Patients* and *Admit Cottage Patient*. We ask you to imagine that a user interface designer has worked on these and produced GUI designs as described in the following paragraphs.

For the *List Cottage Ward's Patients* use case, the user selects a ward name from a scrollable list, and the system displays a scrollable list of the names and ages of the patients on the selected ward as shown by the sketch in Figure 2.

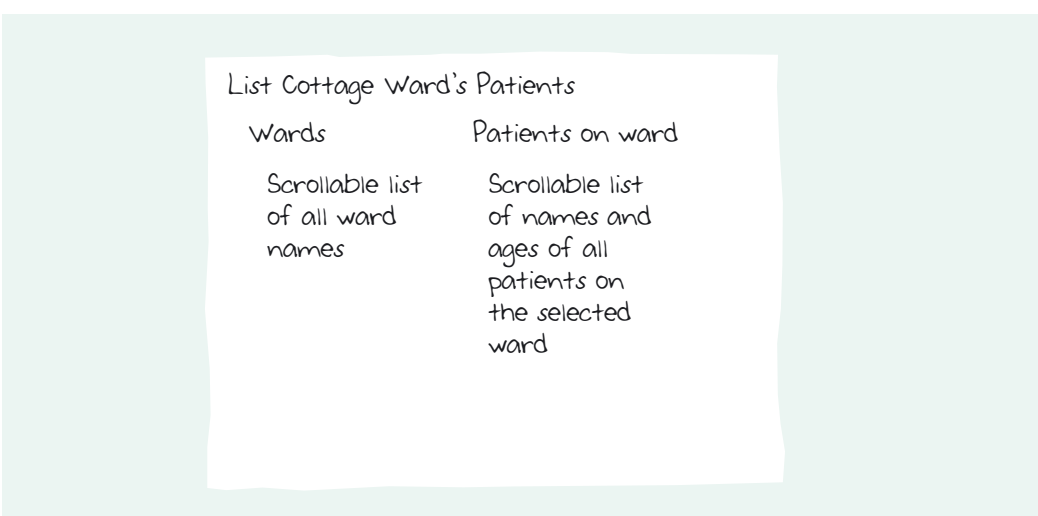


Figure 2 A GUI design for *List Cottage Ward's Patients*

For the *Admit Cottage Patient* use case the user enters the patient's name and date of birth, selects the name of the ward that the patient has to be admitted to, and then clicks a button to record the admission. The result is displayed in a text output area as shown by the sketch in Figure 3.

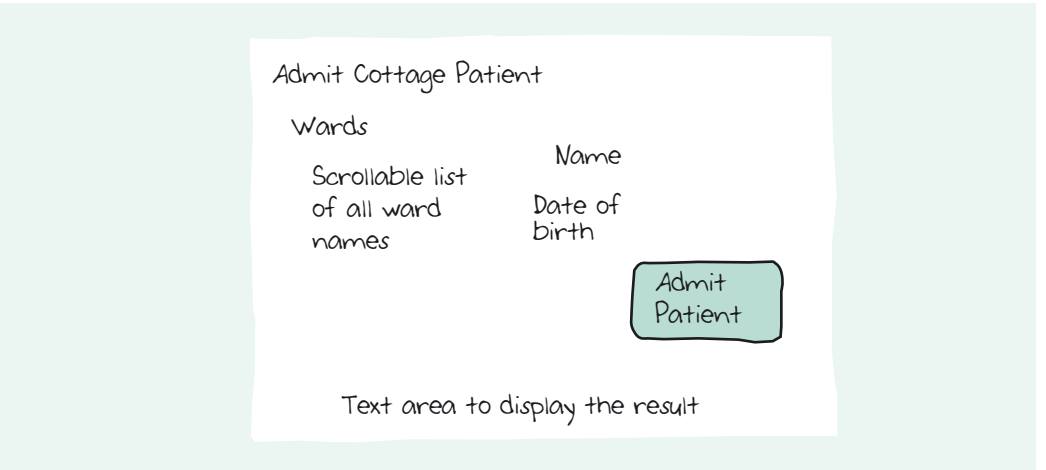


Figure 3 A GUI design for *Admit Cottage Patient*

The imaginary designer has also specified that the part of the GUI for each use case should be positioned on a tabbed screen, as shown in Figure 4.

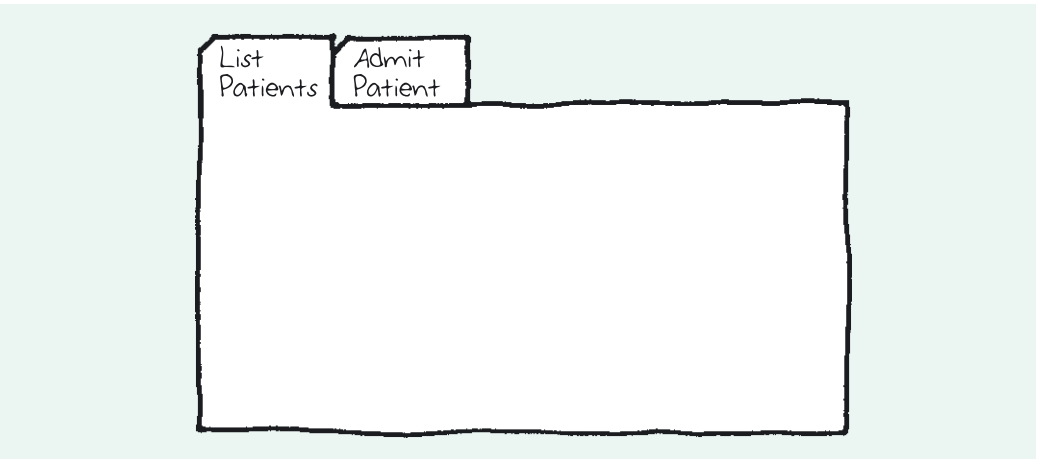


Figure 4 Tabbed screens

However, initially the GUI design for each of the two use cases will be implemented separately; only after this has been done will the interfaces be combined using tabbed screens.

The coordinating class for the core system, `CottageHospCoord`, has two public methods which are the coordinating methods for the two use cases. Here is the Javadoc documentation for these methods.

These two methods are not the only public methods in the core system, as you will see in Exercise 1.

getPatients

`public java.util.Collection<Patient>` **`getPatients`**`(Ward aWard)`

Returns the patients that are on the ward.

Parameters:

`aWard` – a ward

Returns:

a collection of all the `Patient` objects linked to `aWard`

admit

`public void` **`admit`**`(java.lang.String aName, m256date.M256Date aDate, Ward aWard)`

Records the admission of a patient with the given attributes to the ward.

Parameters:

`aName` – the name of the patient

`aDate` – the date of birth of the patient

`aWard` – a ward

Here we have presented the Javadoc documentation rather than the method code, to emphasise the GUI implementer's perspective on the core system. The core system is written to a specification, and the GUI implementer's task is to provide a user interface to it, working to that specification, but without concern for the details of how the core system is implemented. The core system acts as a server, with the user interface being developed as a client.

Taking the use cases from the Cottage Hospital system as examples, we will show how to set up communication between a user interface and the core system, and then how to use a systematic process to build a Java GUI. We will approach the task of implementing the GUI bit by bit, starting very simply and gradually building up the implementation of the designs shown in Figures 2, 3 and 4. Our very first interface will not even be a graphical one! The simplest kind of interface is a textual one, and that is how we will begin. You can think of this as a dry run for implementing a GUI for this use case. It will be an aid to understanding the interactions between the interface and the core system without concern for the details of implementing a GUI.

2.2 Making the first connection

In the following exercise the goal is simply to display a list of the patients on a particular ward, as per the *List Cottage Ward's Patients* use case.

Exercise 1

In NetBeans open the project `Cottage_Hospital_Ex_1`, which is in the folder `M256\M256Code\Exercises\Unit13`. This project contains two packages:

- `cottagehospcore` is a package we have written, in which the core system is implemented;
- `ui` is a package in which you will implement a textual user interface to the core system.

Open the source code for the class `Main` in the package `ui` and inspect the code. Notice that `java.util.*` and `cottagehospcore.*` are imported. The package `java.util` is one of the standard Java libraries containing, amongst many others, the Collection classes.

The class `Main` contains a `main()` method which is empty at present, except for comments. This is where you will be writing the code which will communicate with the core system. The code will be built up in several stages, and the comments summarise what is needed at each stage and show where the code should be inserted.

To generate the Javadoc, from the Build menu choose Generate Javadoc for "Cottage_Hospital_Ex_1", or alternatively right-click on Cottage_Hospital_Ex_1 in the Projects window and select Generate Javadoc for Project.

(a) Generate and inspect the Javadoc for this project, so you can find out about the public methods the core system provides.

As you know, there are three classes in the core system, i.e. `cottagehospcore` contains the coordinating class `CottageHospCoord` and the classes `Patient` and `Ward`, instances of which will represent the patients and wards in the cottage hospital. You need to understand the public methods of these classes, so you should take a little time to browse through the documentation. Notice that `CottageHospCoord` has a single constructor which takes no arguments.

In the body of the `main()` method, add a statement which declares a variable `cottageHospCoord` of class `CottageHospCoord` and assigns a new instance of `CottageHospCoord` to it (that is, assigns a coordinating object to it).

(b) From the Javadoc for the coordinating class, `CottageHospCoord`, you should see that there is a method `getWards()`. Why is this method needed?

(c) For the *List Cottage Ward's Patients* use case the user has to select a ward. In the GUI design it has been decided that this will be done by presenting the user with a list of ward names, but for the simple textual interface being developed at the moment, a simpler means will be used of displaying the ward names.

Write code which uses the method `getWards()` to do the following.

- Obtain a collection containing the `Ward` objects in the core system, assigning the collection to a variable `Collection<Ward> wards`.
- Iterate through `wards`, outputting the name of each ward using `System.out.println()`. The class `Ward` has a method `getName()`, which can be used to obtain the name of each ward.

Once you have added this code, run the project. The names of the wards in the cottage hospital should be displayed.

(d) Although you can iterate through the collection of `Ward` objects, the collection is not indexed in any way and there is no means of accessing individual `Ward` objects, so it is not possible at this point to get a list of the patients on a given ward.

If you need to remind yourself about the class `ArrayList`, or about others as we go along, you will find the API documentation very useful.

To address this you will create an indexed collection of the `Ward` objects. The collection class you will use is `ArrayList`. Since an instance of `ArrayList` has a defined order and can be accessed by index this will let you write code to get a reference to an individual `Ward` object.

To create the list, add the following line to your code.

```
List<Ward> wardList = new ArrayList<Ward>(wards);
```

The `ArrayList` constructor takes the original collection as its argument and uses it to construct an instance of `ArrayList` containing the same `Ward` objects as in the original collection.

The following code will now output the contents of the list in index order.

```
System.out.println(wardList);
```

This `println` statement does what is required because `ArrayList` has a `toString()` method which calls the `toString()` method of its elements, and `Ward` has a `toString()` method which returns the `name` value of a `Ward` object.

When you have added this statement, run the project again, and the list should be displayed.

(e) It is now possible to get a reference to a particular `Ward` object. From the output in part (d), you can tell what position each ward occupies in the list. It can then be accessed using the `List` method `get(index)`, which returns the element at the specified index; for example `get(0)` will return the first element, since the indexing is zero-based.

Add code to do the following.

- Get a reference to the `Ward` object whose `name` is `"Heron"`, and assign it to a variable `Ward theWard`. (You will need to inspect the list of wards from part (d) to find the right index. We found `Heron` at index `3`, but you may have a different index.)

- Send the message `getPatients(theWard)` (this is the coordinating message) to the coordinating object `cottageHospCoord`, assigning the return value to a variable `Collection<Patient> patientCollection`.
- Iterate through `patientCollection`, outputting the `name` and `age` of each `Patient` object. The class `Patient` has methods `getName()` and `getAge()` that can be used for this.

Run the project again, and the list of patients' names and ages should be displayed.

Discussion

A solution to this exercise is in the project `Cottage_Hospital_Ex_1_Sol`.

(a) The following code declares and assigns the required variable.

```
CottageHospCoord cottageHospCoord = new CottageHospCoord();
```

(b) The method `getWards()` is needed so that a user interface can enable the user to select the required ward.

(c) Displaying the ward names is done as follows.

```
Collection<Ward> wards = cottageHospCoord.getWards();
for (Ward eachWard : wards)
{
    System.out.println(eachWard.getName());
}
```

The output will be something like the following, although the order of the `Ward` objects is not predictable and you may get a different order.

```
Lark
Swallow
Nightingale
Heron
```

(d) The final line of the output gives the contents of the list. It will resemble the following, although the order may be different (and might even be different from the result obtained in part (c).)

```
[Lark, Swallow, Nightingale, Heron]
```

(e) Getting the `Patient` objects that are linked to a `Ward` object and outputting their names and ages is done as follows (except that the `3` may be some other number, depending on the position at which the `Ward` object whose `name` is `"Heron"` occurs).

```
Ward theWard = wardList.get(3);
Collection<Patient> patientCollection =
    cottageHospCoord.getPatients(theWard);
for (Patient eachPatient : patientCollection)
{
    System.out.println(eachPatient.getName() + ", "
        + eachPatient.getAge());
}
```

The last part of the output should be as follows.

```
Fred Goldsmith, 29
Herbert Irving, 41
George Harrison, 17
```

(Again the order may vary. The ages will also vary, as they depend on the current date.)

If a browser window did *not* open automatically after Javadoc generation, the Javadoc can be accessed via the `Files` window in NetBeans (viewable by selecting `Windows|Files`). In the `Files` window expand the `Cottage_Hospital_Ex_1` node, next expand the newly revealed `dist` node, and from there expand the `javadoc` node. Right-click on the `index.html` file and select `View`.

Now that you have written code for the interaction between the core system and a very simple textual user interface, it will be helpful to reflect on the interaction that was involved. In summary, leaving out the Java details, the steps were as follows.

1 The user interface obtained a reference to the coordinating object.
2 The interface sent a message to the coordinating object to obtain a collection of all `Ward` objects.
3 A list of all ward names was displayed to the user. This involved the interface sending `getName()` messages to the `Ward` objects.
4 The user selected a ward name.
5 The interface identified the `Ward` object corresponding to the ward name selected by the user.
6 The interface sent a message to the coordinating object to obtain a collection of the `Patient` objects on the selected `Ward` object.
7 A list of corresponding patient names and ages was displayed to the user. This involved the interface sending `getName()` and `getAge()` messages to the `Patient` objects.

It is instructive to look at a sequence diagram illustrating the communication between the user interface and the coordinating object (see Figure 5). As the focus of the diagram is on the communication between the user interface and the coordinating object, the getter messages (`getName()` and so on) that the user interface sends to the `Ward` and `Patient` objects are not depicted.

Unlike the sequence diagrams of earlier units, this sequence diagram is not being used as a design aid. Instead its purpose is to help you understand an interaction that has already been designed and implemented. Sequence diagrams are a good way of explaining object interactions, as you saw in Section 3 of *Unit 1*.

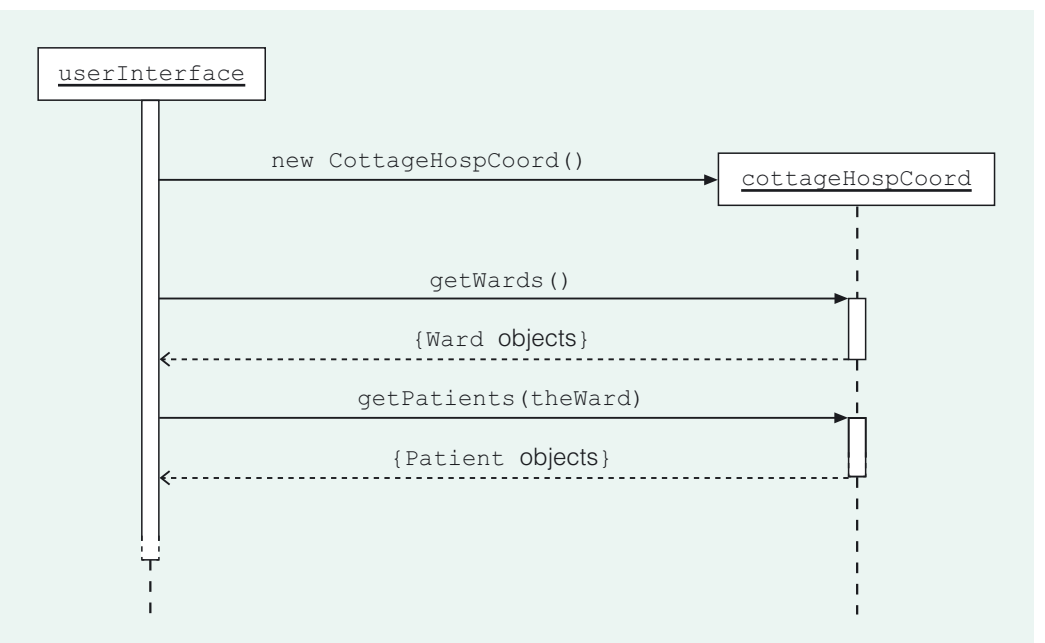


Figure 5 A sequence diagram for *List Cottage Ward's Patients*

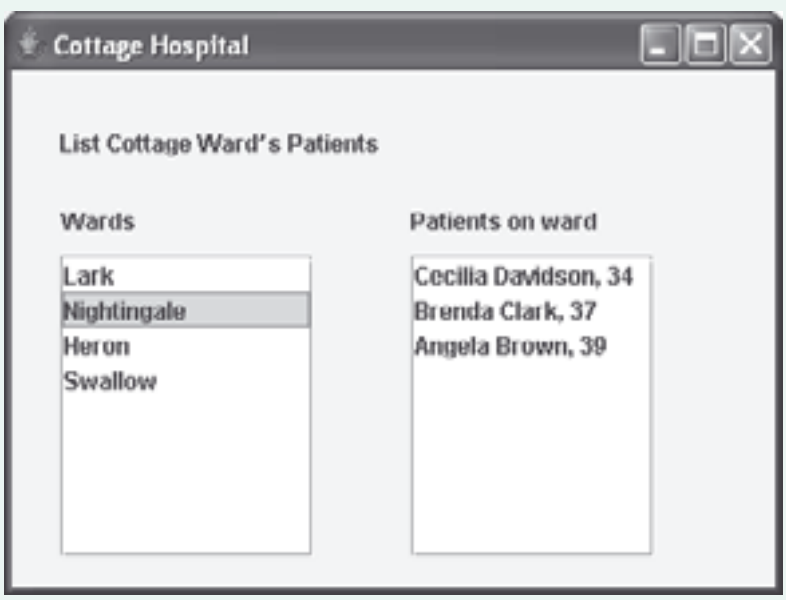
2.3 Creating the first GUI

The interface you have just implemented was a throwaway effort, whose only purpose was to give you a feel for the communication between user interface and core system. It could not conceivably be used in reality – for one thing, the choice of ward is 'hard-wired' into the code whereas in reality the user needs to be able to make a choice interactively.

It would be possible to improve the interface so that it accepted keyboard input, and eventually develop a fully-fledged textual interface. Historically, of course, the early user interfaces were all text-based, since there was no other choice. However the aim in this unit is to build GUI interfaces, and in the next exercise you will see how a GUI interface for the *List Cottage Ward's Patients* use case can be implemented. It is important to recognise though that the underlying communication between the interface and the core system will remain the same.

We will keep the GUI quite simple, and to simplify the task of implementing the visual components you will be using the NetBeans GUI designer.

The finished GUI will look like Figure 6, which is a realisation of the design sketched earlier in Figure 2.



The ages displayed will be dependent on the date the application is run.

Figure 6 A simple GUI for *List Cottage Ward's Patients*

As you saw in *Unit 11*, GUIs are typically built up from various kinds of widget such as labels, lists, buttons, text areas and so on. The Java **Swing** library provides **visual components** which implement widgets, and which can be arranged in a **container** – typically a window of some sort.

Containers are themselves visual components, and so it is possible in Java to nest containers to build up more or less any structure required for a GUI. In a given GUI, one container will be the **top-level container** – the one that contains everything else but is not itself contained.

The user can interact with a widget by clicking on it using the mouse (or by using the keyboard), thereby generating an **event**. The programmer writes code that reacts to an event by carrying out appropriate actions, and in this way the GUI interacts with the user.

This code is called **event handling code** and the method in which such code is written is called an **event handler method**.

This handler method has to be written in a special kind of class that implements a whole category of events that involve the particular widget. Then the programmer adds a **listener** – an instance of the class in which this handler is implemented – to the component. Whenever the user interacts with the widget, the listener picks this up and invokes the handler method written for that event.

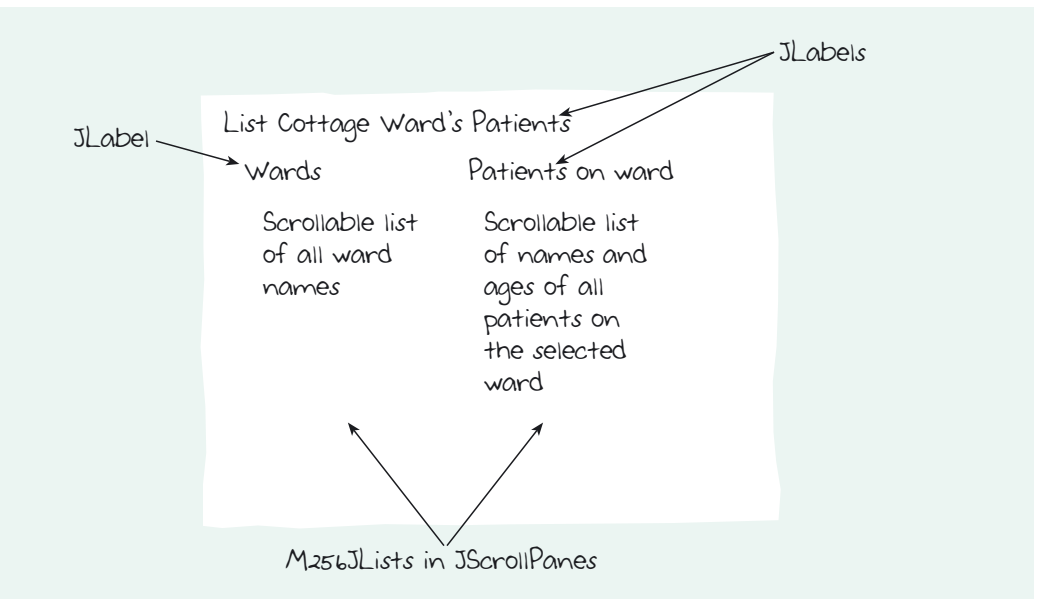
In M256 we only use a few of the more common Swing widgets. These are summarised in the Appendix to this unit, which also briefly explains events and **layout managers**, which are objects that control the way widgets are arranged in a container, and which are provided by the package `java.awt` – the **Abstract Windowing Toolkit** (**AWT**) classes.

To get from the design sketch of Figure 2 to a working GUI, the first step is to decide what visual components are required. When you use the NetBeans GUI designer, an appropriate container – a `JFrame` – is created automatically, but it is up to you to choose what widgets you need and lay them out as required.

In this simple example only a small number of widgets are needed. The design calls for two scrollable list widgets, and there are some captions, which require labels, for each of which we will use a `JLabel`. The standard Swing class for lists is `JList`, but in this unit you will work with a customised version of `JList`, called `M256JList`, which we have provided because it is more convenient for our purposes than the standard class.

For ease of reference we will typically write, for example, 'a `JLabel`' rather than 'an object of the class `JLabel`'.

In Figure 7 we have reproduced the design sketch of Figure 2 and marked on it the widgets we have chosen.



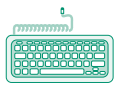
The figure shows that to obtain a scrollable list, `M256JList`s will be used inside `JScrollPane`s (you will learn about this in Exercise 2(d)).

Figure 7 The widgets which will be used for *List Cottage Ward's Patients*

You should now be prepared for the next exercise, which requires you to implement the GUI design for *List Cottage Ward's Patients*. This exercise leads you through a number of steps in the implementation of a screen for this use case, telling you what to do and asking you to run the evolving implementation to check whether each step is correct.

Before beginning this exercise you should have worked through Section 5 of the *NetBeans Guide*, as noted in the Introduction to this unit. You may find it useful to have the *NetBeans Guide* to hand for reference as you work.

Exercise 2



In NetBeans open the project `Cottage_Hospital_Ex_2`, which is in the folder `M256\M256Code\Exercises\Unit13`.

This project contains two packages:

- `cottagehospcore` is the core system package, as before;
- `gui` is a package in which you will implement a GUI interface to the core system for the *List Cottage Ward's Patients* use case.

(a) **Create a new form**

In the `Projects` window expand the `Cottage_Hospital_Ex_2` project node, then the `Source Packages` node. Right-click on the `gui` package node and choose `New|JFrame Form...`. In the `New JFrame Form` dialogue box, name the class `ListWardsPatientsScreen` and make sure the package name is correctly entered as `gui`. Click on `Finish`. The NetBeans GUI designer will open, with a similar appearance to Figure 8.

NetBeans uses 'design' in this context to mean 'implement'.

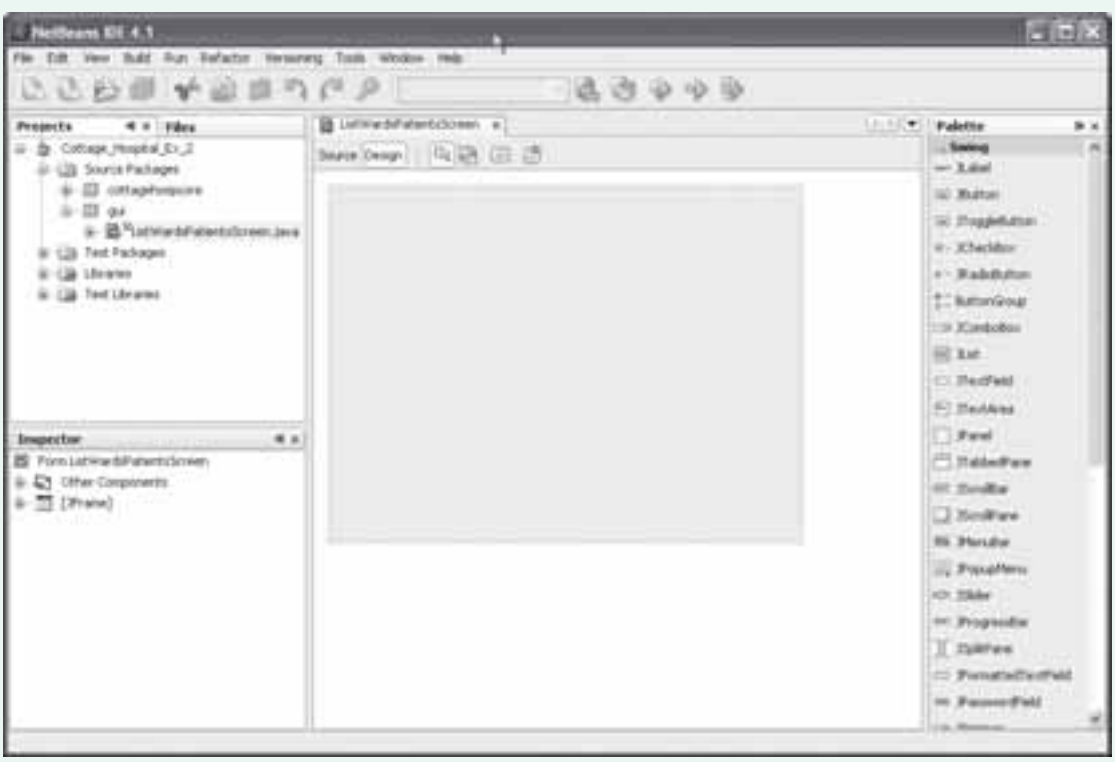


Figure 8 The NetBeans GUI designer

The shaded rectangle in the middle is the Form Editor, in which you will create the GUI. The container for the GUI will be the `JFrame`, empty at present, which NetBeans has created for you. When you add visual components in the Form Editor, NetBeans will automatically generate code to add the corresponding Swing objects to the `JFrame`.

The `JFrame` is represented in the `Inspector` window (bottom left in Figure 8), which contains a tree view of the GUI so far. As you add new components, they will appear in the `Inspector` window.

As well as the node representing the `JFrame`, and the one above it that represents the design as a whole, there is a node entitled `Other Components`. This is a receptacle for any non-visual components that are added, although we shall not be making any use of this facility in M256.

An example of a non-visual component, which would not itself appear on screen, is a timer. The design might have a visual clock display, which would need to be backed up by an unseen timer that would actually keep track of the time.

(b) **Set the form properties**

In the Inspector window right-click on [JFrame] and choose Properties from the menu that drops down. This opens a dialogue box in which you can set the properties of the JFrame. You may need to resize this dialogue box as sometimes its default size is too small. If the Properties button at the top of the dialogue box is not highlighted, click on it to bring up the Properties dialogue box.

The first thing you will do is set the title of the JFrame. Locate the title property – the second item from the top – and in the empty field on the right type Cottage Hospital. Press Enter on your keyboard, to save the title, before continuing.

Next click on the button named Code. Next to the third item down – Form Size Policy – you will see the policy Generate pack(). A different policy is required here (you do not need to understand why), so click on the down arrow button to the right and select Generate Resize Code. Click the Close button to exit the Properties dialogue box.

The layout of a container controls how widgets can be arranged within it.

Finally you must set the layout of the JFrame. This is not done using the Properties dialogue box but by a different route. In the Inspector window, right-click on [JFrame] and choose Set Layout|Null Layout. This type of layout will allow you to position Swing components anywhere you like on the JFrame.

You can test what you have done so far by running the project. If you are invited to set a main class choose gui.ListWardsPatientsScreen. A blank window should appear, with the title Cottage Hospital.

Close the window in the normal way, by clicking on the close icon  .

(c) **Add the labels**

If a different palette is displayed, consult the *NetBeans Guide* to find out how to obtain the Swing palette.

To the right of the Form Editor is the Palette window, from which you can select Swing components to drop onto the Form Editor.

Next you will add the labels. One of the items in the palette should be JLabel. Click once on this, then click anywhere in the Form Editor. A label will appear (Figure 9). Notice that the label has also been added to the tree view in the Inspector window.

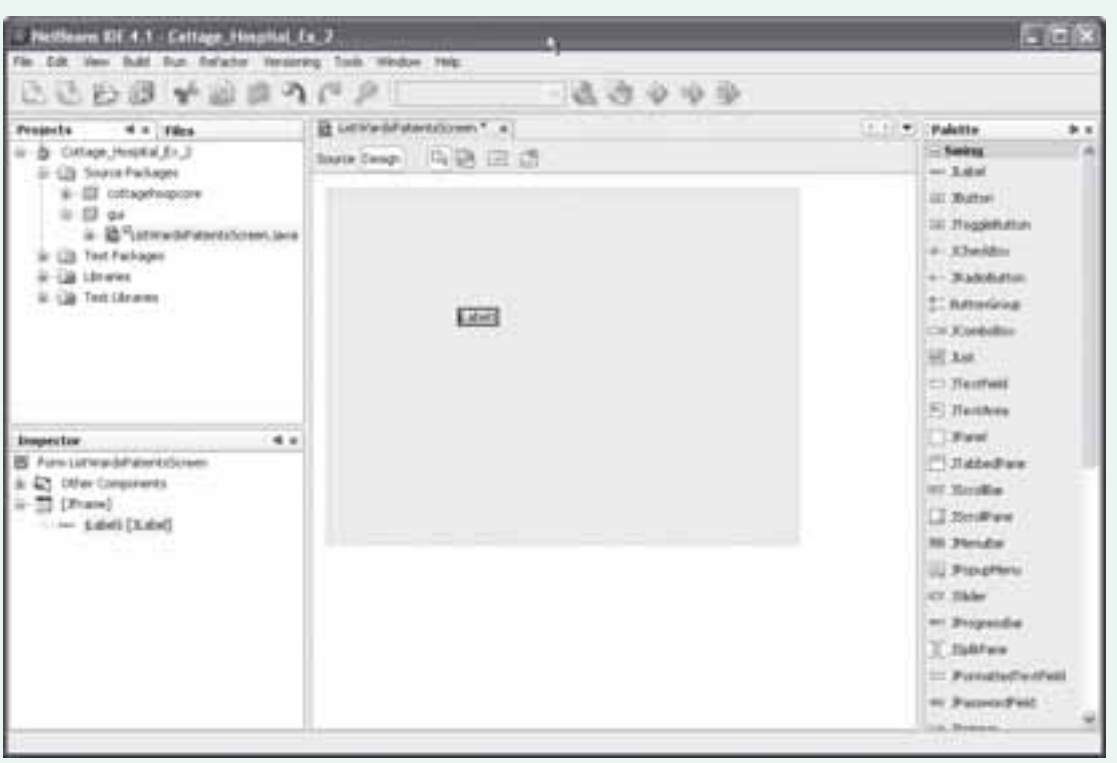


Figure 9 A label is added

Drag the label to a position similar to that of 'List Cottage Ward's Patients' in Figure 2, and resize it so that it is large enough to contain the text 'List Cottage Ward's Patients'. You need not be too concerned about the location and size of the label for now: if necessary, these can be modified when you have tested it.

Right-click on the label in the Form Editor (or on the node that represents it in the `Inspector` window – either will do). Choose `Edit Text` from the menu. The appearance of the label will change, and the text will be highlighted. Edit it to read `List Cottage Ward's Patients`, and click on the design anywhere outside the label. Now run the project again: the label should be displayed.

You can also edit the text in a label by clicking on the text itself. The text will be highlighted and can be edited.

Each scrollable list (which you will add in step (d) below) should have an associated label to say what its contents are (see Figure 2). Add these labels (`Wards` and `Patients on ward`) now, following the same procedure as you used above.

Run the project again. All three labels should be displayed.

(d) **Add the lists**

In Figure 2 two lists are displayed. On the left is a list of ward names; on the right is a list of the names and ages of the patients on the selected ward. The lists must be linked: if a different selection is made from the list of wards, the list of patients must change correspondingly.

Implementing the lists involves three processes:

(i) adding two empty list components to the `JFrame`, using the GUI designer;

(ii) writing code to populate the left-hand list with ward names;

(iii) writing code to detect when a new ward selection is made, and to display the corresponding patient information in the right-hand list.

You will now address the first process in this step, while the other two will be dealt with in steps (e) and (f) below.

Before adding the two empty lists, consider the fact that, if a list is longer or a list item wider than the rectangle allocated to it, then both dimensions should scroll so that all the information can be viewed. This is a feature of the design in Figure 2.

To obtain a scrollable list you first need to add a component known as a **scroll pane** and then add the list to the scroll pane, thus ending up with a scrollable list. This list will then display scrollbars as and when they are required. If there are too many items, a vertical scroll bar will automatically appear. If an item is too wide, a horizontal scrollbar will automatically appear.

To add a scroll pane, click on `JScrollPane` in the palette (you may have to scroll down to locate it), then click once in the Form Editor. The `JScrollPane` will appear, at first as a dot, which can be dragged and resized. When you are satisfied with its position and size (using Figure 2 as a guide), the next step is to add the list to it.

For the list itself we will use an instance of the class `M256JList` that was mentioned earlier. `M256JList` should be available on the palette along with the other visual components.

Click on `M256JList` in the palette and then click on the `JScrollPane` in the Form Editor. The `M256JList` should be added to the `JScrollPane`. You can tell if this has worked correctly from the `Inspector` window, which should show that an `M256JList` called `m256JList1` is nested within a `JScrollPane` called `jScrollPane1` (see Figure 10).

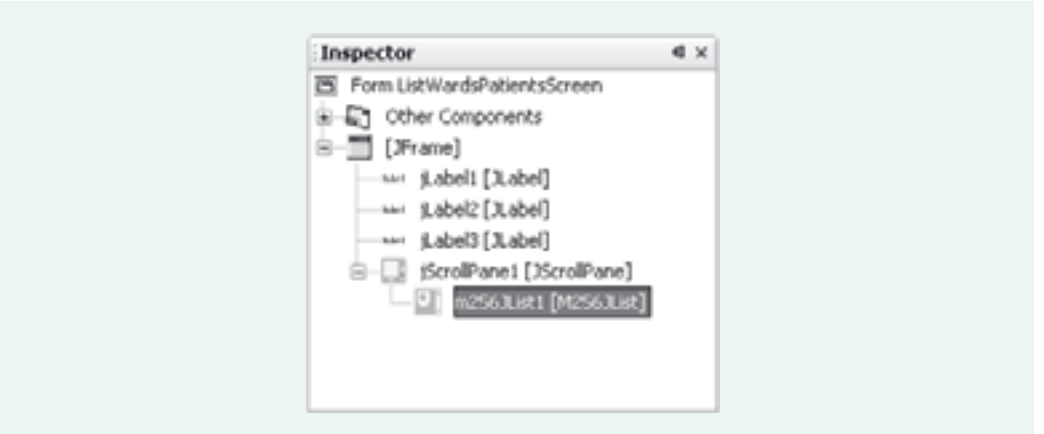


Figure 10 An `M256JList` has been added to the `JScrollPane`

If you need to remove an item, right-click on the node that represents it in the `Inspector` window, and choose `Delete` from the menu.

If the structure is different – for example if the `M256JList` has accidentally been added to the `JFrame` itself – remove the `M256JList` and try again.

Once you have successfully added your scrolling list, run the project to check what you have done so far.

Now follow exactly the same process to add the second scrolling list.

Having added the two `M256JList` objects you should now rename the variables which refer to them. NetBeans gives them the default names `m256JList1` and `m256JList2`, but remembering which is which will be difficult unless more meaningful names are used. It will be necessary to send messages to the lists to set their contents and in the case of the list of wards to find out which ward the user has selected, so working with the default names would be confusing.

In the `Inspector` window, right-click on `m256JList1` and choose `Rename...` from the menu. In the dialogue box that appears enter the `New Name` as `wardList` and click on `OK`. The name will change in the `Inspector` window.

Using the same method, rename `m256JList2` as `patientList`.

The variables referring to the two `JScrollPane` widgets and the `JLabel` widgets could also be renamed, but this is not essential because the objects concerned will not be sent any messages.

At this point you should run the project again. If everything is correct the GUI should resemble Figure 11.

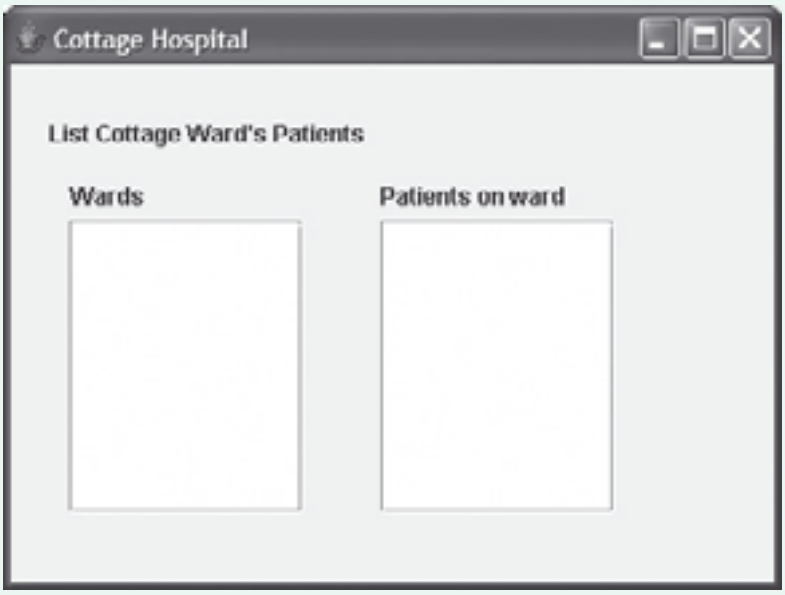


Figure 11 The basic GUI for *List Cottage Ward's Patients*

(e) **Populate the list of ward names**

The next task is to display the names of the wards. This is the point at which you will write the code to make a connection to the Cottage Hospital core system.

Click on the `Source` button that appears just under the tab labelled `ListWardsPatientsScreen`. This will open a Source Editor for the class `ListWardsPatientsScreen`, which, as you can see from the source code, extends `JFrame`. The code for this class is the code that actually implements the GUI. The Form Editor allows you to edit this class visually using drag and drop, and that is how you have constructed the GUI up to now, with all the corresponding code being generated automatically by NetBeans. For the next step, however, you will need to edit the source code by hand.

Notice that while the Source Editor is open the `Inspector` window is temporarily hidden, and reappears when you return to the Form Editor.

The GUI will need to use classes from the Cottage Hospital core system as well as Collection classes (just as the textual interface in Exercise 1 did), so begin by adding the following import statements just before the class declaration.

```
import cottagehospcore.*;
import java.util.*;
```

The coordinating object will need to be accessible to the code in the GUI class `ListWardsPatientsScreen`, so you will need to add code declaring it as an instance variable. This should go immediately after the class declaration as follows.

```
public class ListWardsPatientsScreen extends javax.swing.JFrame
{
    private CottageHospCoord cottageHospCoord;
    ...
```

Next locate the constructor, which reads as follows.

```
/** Creates new form ListWardsPatientsScreen */
public ListWardsPatientsScreen()
{
    initComponents();
}
```

You can view the contents of `initComponents()` if you wish, but you should not try to edit it because doing so may prevent the GUI from working correctly.

When a new `ListWardsPatientsScreen` object is created the method `initComponents()` is executed. This is a method that has been generated automatically by NetBeans, and is responsible for creating all the visual components

and adding them to the GUI. Every time the design is changed, the code of `initComponents()` is altered accordingly by NetBeans.

You will now add code to the constructor to obtain a collection of `Ward` objects from the core system and display their names in the GUI. The interaction will be similar to that in Exercise 1. All the statements you insert should come after the statement `initComponents();`.

The first statement you should add obtains a reference to the coordinating object.

```
cottageHospCoord = new CottageHospCoord();
```

The second statement to add asks the coordinating object for a collection of the `Ward` objects.

```
Collection<Ward> wards = cottageHospCoord.getWards();
```

Similar statements appeared in Exercise 1. The unfamiliar part consists of the next two statements, which:

You need know nothing more about the `setListData()` method.

(i) add the collection of `Ward` objects to the `M256JList` object, `wardList`, using the `setListData()` method;

(ii) set the first element in the list to be selected by default when the list is first created, using the `setSelectedIndex()` method.

The required statements are

```
wardList.setListData(wards);
wardList.setSelectedIndex(0);
```

When you have added these statements, run the project again. The names of the wards should be displayed in the GUI as shown Figure 12.

The order in which the ward names appear is not predictable, and your wards may appear in a different order from ours.

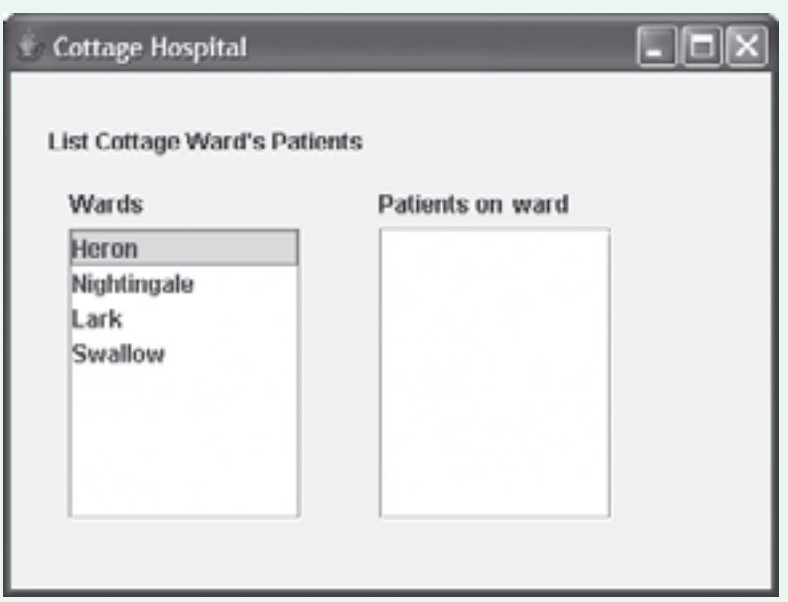


Figure 12 The names of the wards are displayed

Notice that, although the names are what actually appears, it was the actual *objects* that were added to the `M256JList`. To display each object, the `M256JList` invokes `toString()` on the object to produce a string representation, which in the case of a `Ward` is its name (because the `toString()` method of `Ward` simply returns the `name` value).

This will not work for the patient list, because the `toString()` method of `Patient` returns information about `name` and `dateOfBirth` whereas the list is required to display the name and age of each patient. You will see in a moment how to deal with displaying the patient list.

(f) **Display the details of the patients on the selected ward**

So far the GUI does not respond to events such as mouse clicks. If you run the project and select different ward names, nothing happens.

In this final step you will add code that listens for the event triggered by a user selecting a ward name and, in response, displays the corresponding list of patient details. This event is a *list selection* event, because the user is selecting a list item. More specifically, it is a *value changed* event, because the act of clicking on a ward name changes which value in the list is the one currently selected. (If you click on the same ward name as before there will be no actual change, but a value changed event will still be generated.)

Click on the `Design` button to leave the Source Editor and return to the Form Editor. The `Inspector` window will reappear. (If it does not, choose `Window|GUI Editing|Inspector`.)

In the `Inspector` window right-click on `wardList` and choose `Events|ListSelectionListener|valueChanged`. This will attach a listener to `wardList` and automatically open the Source Editor at the location where you can write the code specifying what should happen when a value changed event occurs. This location is shown by the highlighted portion of source code in Figure 13.

Most events can cause several possible messages to be sent to listener objects, depending on the precise nature of the event. For example, a mouse event can result in any of `mouseClicked`, `mouseEntered`, `mouseExited`, `mousePressed` and `mouseReleased`. When adding an event listener in NetBeans, you must first choose the kind of event, then the particular message you want sent to the listener. For example, if you require a listener to respond to mouse clicks, you choose `Mouse|mouseClicked` from the menu.

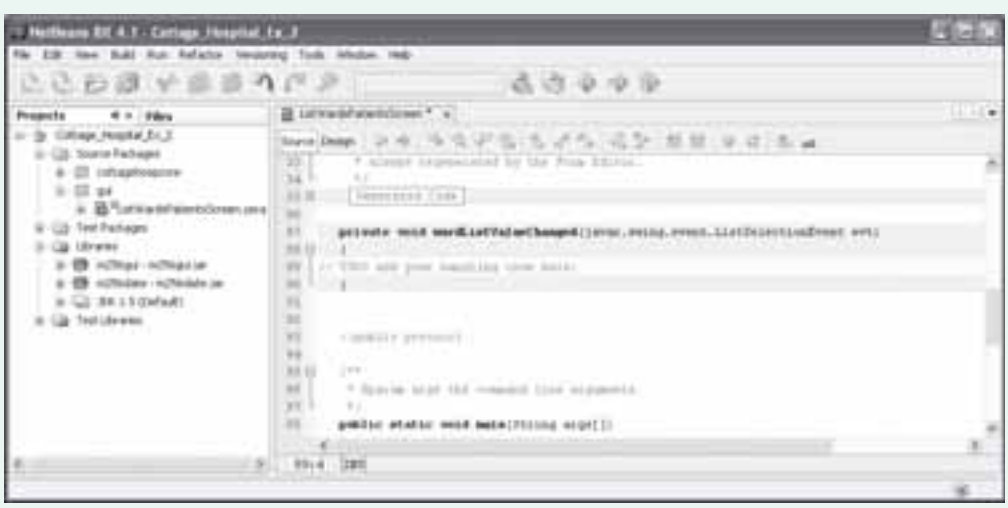


Figure 13 Specifying the response to a value changed event

When the event occurs, the handling code you are about to write will be invoked. This handling code needs to do four things:

(i) find out from `wardList` the `Ward` object corresponding to the ward name selected by the user;

(ii) ask the coordinating object for the collection of `Patient` objects linked to that `Ward` object;

(iii) extract the required details from each of the `Patient` objects and build a new collection of the data that needs to be displayed;

(iv) set the list data of `patientList` to be the collection of patient data to be displayed.

To perform the first step, the `getSelectedValue()` method is used. This returns an `Object`, so a cast to `Ward` is required.

```
Ward theWard = (Ward)wardList.getSelectedValue();
```

For the second step, you saw how to obtain the collection of `Patient` objects in Exercise 1.

```
Collection<Patient> patients =
    cottageHospCoord.getPatients(theWard);
```

Add both these statements to the handling code now.

To implement the third step, you need to create a new collection of `String` objects, and then iterate through the `Patient` objects, adding the details (`name` and `age`) of each `Patient` object to the new collection. You will use an `ArrayList` to hold the patient details, although various other kinds of collection could equally well have been used instead. Notice that in the following code, which you should now add to your handling code, the patient details are extracted in exactly the same way as in Exercise 1.

```
List<String> patientData = new ArrayList<String>();
for (Patient eachPatient : patients)
{
    patientData.add(eachPatient.getName() + ", " + eachPatient.getAge());
}
```

Finally, add the patient data to `patientList`, using the `setListData()` method that you saw previously:

```
patientList.setListData(patientData);
```

Now run the project again. Click on a ward name and details of its patients will appear in the `Patients on ward` list. For example, if you click on `Nightingale`, your GUI should look similar to that in Figure 14.

The ward names may appear in a different order from previously.

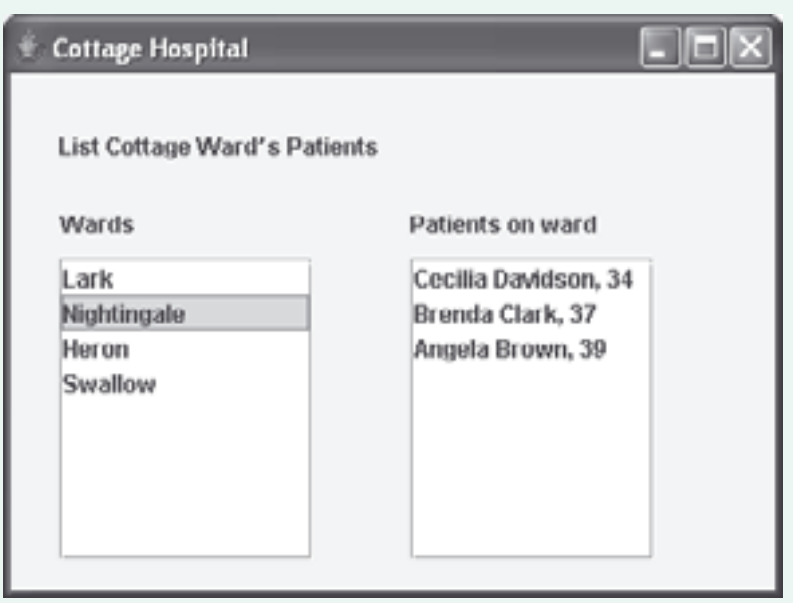


Figure 14 The patients on a selected ward

Discussion

There is no extended discussion for this exercise. However you can compare what you have done with our answer, which is in the project `Cottage_Hospital_Ex_2_Sol`.

2.4 Reflecting on interfaces

Component-based GUI development

In implementing a GUI interface you have been putting into practice the ideas of component-based development that were introduced in *Unit 5*.

The palette lets you select items from a collection of predefined building blocks – the Swing components, along with an additional M256-specific component. The work of designing and implementing these has already been done, and you can just take them 'off the shelf' and use them. Note the following characteristics, typical of reusable components.

- Each type of component has a well-defined protocol – the set of messages that it will respond to, corresponding to methods of the component.
- The components are documented, so the developer (you) can know what the component does and how to use it. The documentation specifies how to invoke each method and what result can be expected. For example, the Javadoc for `JLabel` describes the purpose of a `JLabel` object and gives some general guidance about its use; then, for each method defined in the class, it specifies what the method does, what arguments it requires and what value it returns.
- The user of a component does not have to be concerned with how a component is implemented, but works entirely from the documentation. This means that the component can be replaced by one with a different implementation, and as long as the replacement meets the same specification the user will not be affected.
- Components can be nested inside other components to form a hierarchy. The way in which the GUI is built up from components within components is displayed in the `Inspector` window.
- The components are adaptable. To implement a GUI the implementer picks the components that are required for a particular design, drops them into a suitable structure, and adapts them by writing the code that makes them interact as required.

Different interfaces, same core system

In this unit so far you have seen two different kinds of user interface. Although we did not pursue the idea, it would be perfectly feasible to develop a purely textual interface, which accepted keyboard input and displayed all output as text on the computer screen. This is in contrast to the user interfaces which are the main concern of this unit, which are graphical user interfaces. Other quite different forms of interface are also possible. For example, it is conceivable to have an interface that works entirely by sound, responding to spoken commands and outputting results as synthesised speech.

As you have learnt in earlier units, this ability to attach different interfaces, and interfaces of different types, to exactly the same underlying core system is one of the great advantages arising from keeping the core system and the user interface as separate as possible.

The other big advantage is that the implementation of the core system could change without affecting the interfaces. Because the above interfaces have been produced working solely from the published documentation for the core system – its Javadoc – without making any assumptions about how it is actually implemented, the core system could change, and so long as it continued to meet the same specification, the interfaces would continue working.

In this section you have developed two user interfaces for the simple Cottage Hospital System. The first one was a simple textual interface which was designed to clarify how an interface communicates with the core system, while the second built on this knowledge in using NetBeans to implement a GUI interface. Implementing a GUI was identified as similar to component-based development as described in *Unit 5*. This section also briefly reviewed the benefits of developing user interfaces that are independent of the core system.

3 Building a GUI in a systematic way

When you built the GUI for *List Cottage Ward's Patients* in the last section, hardly any planning was involved: the implementation was based solely on a simple design sketch. This informal approach was possible because the use case concerned involved only a very simple screen and just a single user interaction. For most use cases the GUI will be more complex, and its construction, like any other part of software development, will require a more disciplined process.

In this section you will:

- learn how to build a GUI systematically, starting from a design sketch;
- put the process into practice with an example use case;
- see how a GUI can be extended to cope with invalid input data.

3.1 A systematic approach

In a real project several people may be involved in developing the GUI, and in particular the GUI designer and the GUI implementer may be different people. The design passed to the implementer by the designer will specify the layout of the GUI as a series of sketches (storyboards). It will include details such as:

- what information should be displayed on each screen;
- what widgets are required, and how they should respond to user actions.

However, the design will not specify exactly how the GUI should be implemented in Java, or any other language for that matter. If an appropriate design process has been employed, it will have focused mainly on the users and how their needs can be met, not on how the interface is going to be coded. It is then up to the implementers of the GUI to translate the design into working Java code. They need to plan for this by considering a number of questions, such as the following.

- What Java components should be used to implement the widgets shown in the design?
- Which components need to be nested inside other components, and what will the overall component hierarchy look like?
- How should the components be initialised?
- What events do the components need to respond to?

Below, we outline a structured process for building a GUI that addresses these kinds of questions:

A Draw up a plan.

- A1 Starting with the GUI design sketch and the description of the use case, decide what components need to be used and whether any need to be nested inside other components.
- A2 Identify any component properties that need to be set differently from the default, and decide on any layouts required.
- A3 Determine which components must be initialised and with what data.
- A4 Identify what interactions are needed – what events can occur and what the responses should be.

B Translate the plan into a GUI.

B1 Create a form to contain the GUI.

B2 Add the components.

B3 Set the required properties.

B4 Write code to obtain a reference to the coordinating object.

B5 Write code to initialise components.

B6 Add events to the widgets and write the event handling code.

This may look a little complicated, but in fact it simply encapsulates the steps followed in the previous section. It is a useful checklist, though you are not expected to memorise it: the important thing is to get a feel for the steps involved in building a GUI.

To demonstrate how the process outlined above is used, it will be applied to the *Admit Cottage Patient* use case. When we introduced this use case earlier, we only described it informally. Here is a more precise description.

> The user provides the patient's name and date of birth, and identifies the ward that the patient is to be admitted to. The system records that the patient is on the ward.

For ease of reference, Figure 15 repeats the design sketch that was shown earlier in Figure 3.

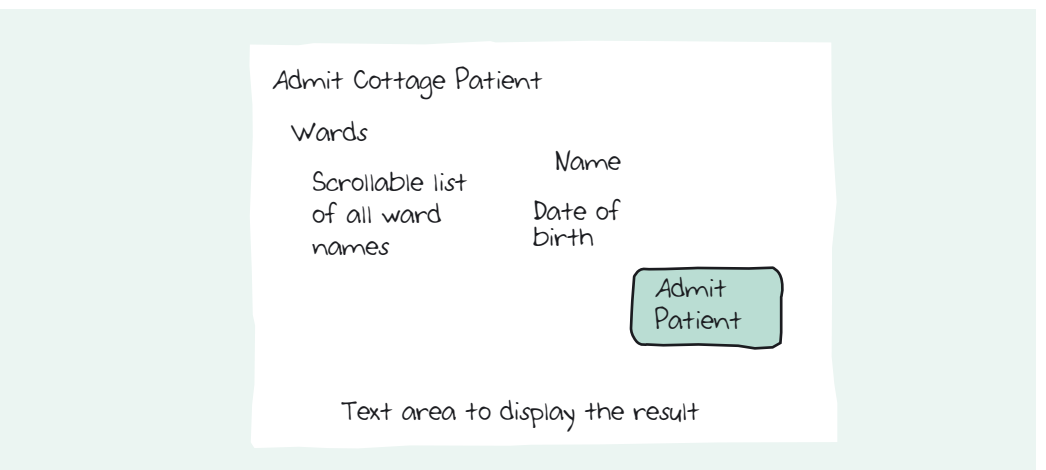


Figure 15 The GUI design for *Admit Cottage Patient*

In terms of the GUI, the use case involves the following actions.

The user will:

enter the patient's name and date of birth into the appropriate text fields;

select a ward from the scrollable list of ward names;

press the `Admit Patient` button.

The system will:

display the result of the user's actions in the text area.

It will also be necessary to refer to the Javadoc for the coordinating method, so here it is again.

admit

```
public void admit(java.lang.String aName, m256date.M256Date aDate, Ward aWard)
```

Records the admission of a patient with the given attributes to the ward.

Parameters:

`aName` – the name of the patient

`aDate` – the date of birth of the patient

`aWard` – a ward

As the GUI implementer, you are now ready to apply the GUI building process outlined above to the *Admit Cottage Patient* use case.

3.2 Applying the systematic approach

A Draw up a plan

Step A1

In Subsection 3.1, this step was described as follows:

> Starting with the GUI design sketch and the description of the use case, decide what components need to be used and whether any need to be nested inside other components.

Exercise 3

Considering Figure 15, and drawing on your experience from the activities earlier in the unit, decide on the different types of component you would use. (You may also like to refer to the Appendix.) You should record your decisions by making a sketch of the screen in Figure 15 and annotating it with the relevant components, in the manner of Figure 7.

Discussion

The components we selected are shown in Figure 16. You might have used `JTextArea` instead of `JTextField`, and you may also have labelled the outer container as a `JFrame`.

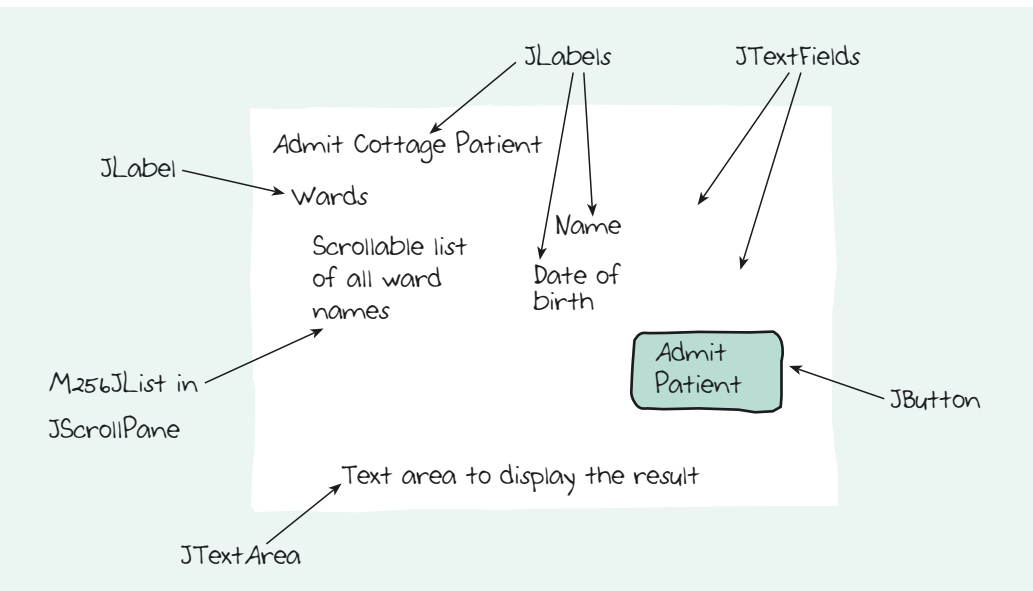


Figure 16 The components that the GUI will use

Since the screen divides naturally into an upper half containing the widgets the user interacts with and a bottom half in which the system displays feedback, you may also have considered using upper and lower `JPanel`s and nesting the other widgets in these two panels. With a more complex GUI you would certainly need to use nested panels, but in this case we decided to keep everything as simple as possible and place the widgets directly on the `JFrame` (apart from nesting the `M256JList` in `JScrollPane` of course).

Next you must consider how the components are nested.

Exercise 4

In Figure 17 we have started sketching a tree view of the GUI, approximately as it might appear in the `Inspector` window, but simplified a bit. We have not drawn icons, but just given variable names and the corresponding classes. The sketch is not finished yet: four components are missing from it.

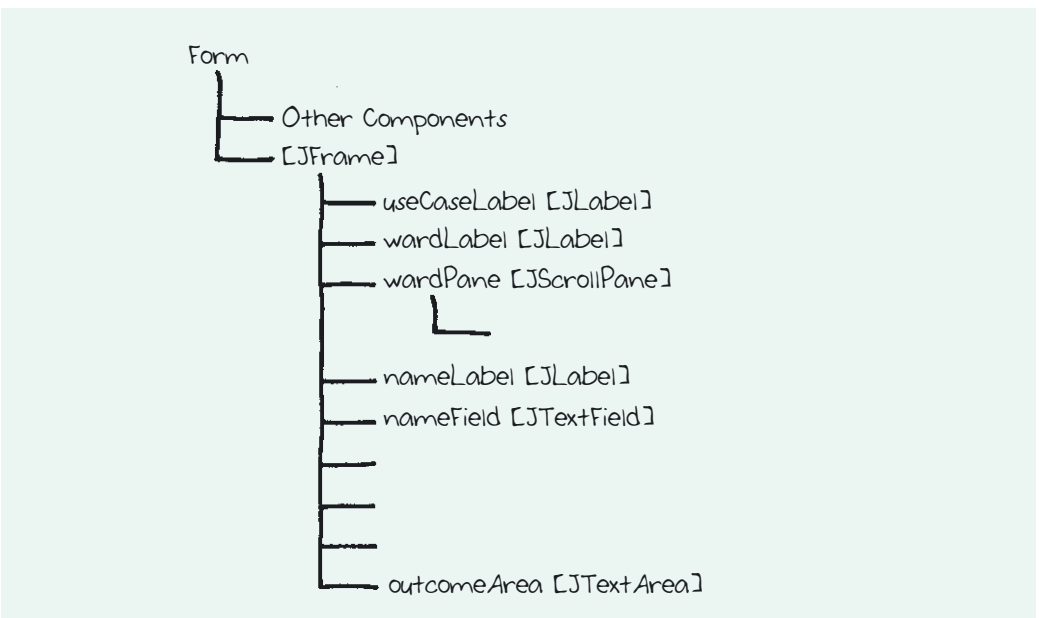


Figure 17 An incomplete tree view of the GUI for *Admit Cottage Patient*

As you have learnt, it is not generally necessary to give meaningful names to components such as labels, but here it helps in the planning process.

On paper, make a copy of this sketch and fill in the missing components, giving each a meaningful name and indicating what class it belongs to.

Discussion

Figure 18 shows our sketch; you may have used slightly different names or listed some of the components in a different order.

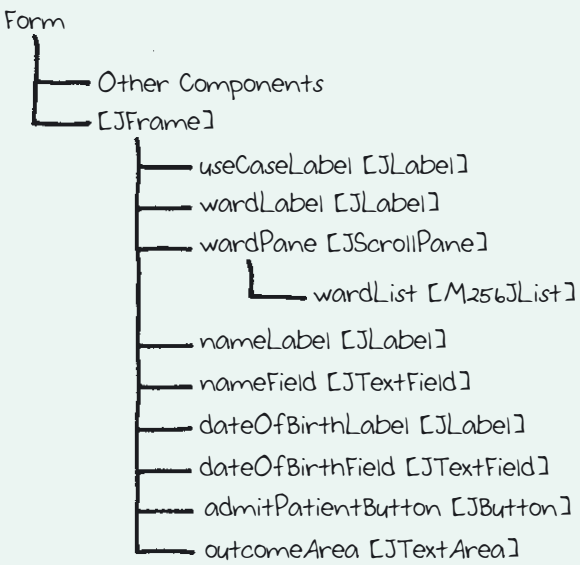


Figure 18 A tree view of the GUI for *Admit Cottage Patient*

Step A2

In Subsection 3.1, this step was described as follows:

> Identify any component properties that need to be set differently from the default, and decide on any layouts required.

A typical GUI design would require many component properties to be set. A widget's visual appearance, for example, is typically largely controlled by the corresponding component's properties, and a sophisticated design would specify in detail the appearance of each widget (font, colour, border style etc).

In our simple example, very little needs to be done. Because the design sketch shows a horizontal line separating the `JTextArea` at the bottom of the window – the output area (see Figure 16) – from the rest of the design, we will sets its `border` property to `EtchedBorder`. We will also set its `editable` property to `false`: given that this area is for system messages, the user should not be able to type in it.

The layout of the `JFrame` needs to be set to `NullLayout` as before, and the Form Size Policy to `generateResizeCode()`. The `title` is `Cottage Hospital`.

Although there are only a few property settings and layouts to consider, we have summarised them in Table 1. In a more complex case, making such a record would be essential, because without a systematic means of keeping track there would be little chance of getting all the properties set correctly.

Table 1 Component properties

Component	Class	Property	Value
Main frame	`JFrame`	layout	`NullLayout`
		Form Size Policy	`generateResizeCode()`
		`title`	`Cottage Hospital`
`outcomeArea`	`JTextArea`	`border`	`EtchedBorder`
		`editable`	`false`

Step A3

In Subsection 3.1, this step was described as follows:

> Determine which components must be initialised and with what data.

Only two components need to be initialised.

1 The `M256JList` referenced by the variable `wardList` must be populated with a list of `Ward` objects whose `name` values will be displayed.

2 The `JTextArea` at the bottom of the window might be left empty initially. However the user will find it useful to have an indication of what this area is for, so it will be initialised with the `String` value `"Outcome:"`.

Again a complex design might require the initialisation of many components, so as part of a systematic process it is useful to draw up a table of initialisation information (see Table 2).

Table 2 Initialisations

Component	Class	Description of data	Type of data
`wardList`	`M256JList`	List of `Ward` objects	`Collection<Ward>`
`outcomeArea`	`JTextArea`	`"Outcome:"`	`String`

Step A4

In Subsection 3.1, this step was described as follows:

> Identify what interactions are needed – what events can occur and what the responses should be.

Assuming the user has entered the patient's name and date of birth and selected a ward, then when the user clicks on the **`Admit Patient`** button, the *Admit Cottage Patient* use case should be initiated.

A `mouseClicked` event could be used instead of an `actionPerformed` event.

Because the event is triggered by clicking on a button, the event will be an `actionPerformed` event, and an `ActionListener` will be attached to the button `admitPatientButton`. When the user clicks on the button, the event handling code should gather the required arguments and send a coordinating message `admit(aName, aDate, aWard)` to the coordinating object. At this stage it is assumed that the user has provided the patient details. Later we will consider what should happen if this is not the case.

There is only one event in this use case, but in general a GUI will have to respond to many different events, so it is useful to draw up another table (see Table 3).

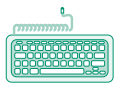
Table 3 Events

Component	Class	Event	Action to be taken
`admitPatientButton`	`JButton`	`actionPerformed`	Send coordinating message to the coordinating object.

This completes the planning of the interface. The next exercise requires you, as the implementer, to implement it.

B Translate the plan into a GUI

Exercise 5



In NetBeans open the project `Cottage_Hospital_Ex_5`, which is in the folder `M256\M256Code\Exercises\Unit13`. This project has a similar structure to previous projects, i.e. it contains two packages:

- `cottagehospcore` is the core system package;
- `gui` is a package in which you will implement a GUI interface to the core system for the *Admit Cottage Patient* use case.

The `AdmitPatientScreen` class in the package `gui` contains a partial implementation of the GUI which was planned above. Steps B1–B3 from the procedure outlined in Subsection 3.1 have already been completed. That is, the form that contains the interface components has been created, all the components have been arranged in it and the necessary properties have been set. The following steps remain:

B4 Write code to obtain a reference to the coordinating object.

B5 Write code to initialise components.

B6 Add events to the widgets, and write the event handling code.

Your task is to carry out these steps and so complete the GUI. All the design work with the Form Editor is complete, and what you are asked to do involves editing only the code of the `AdmitPatientScreen` class in the Source Editor.

Use the following steps as a guide to completing steps B4, B5 and B6.

(a) **Begin by running the project**

The incomplete GUI should be displayed as shown in Figure 19.

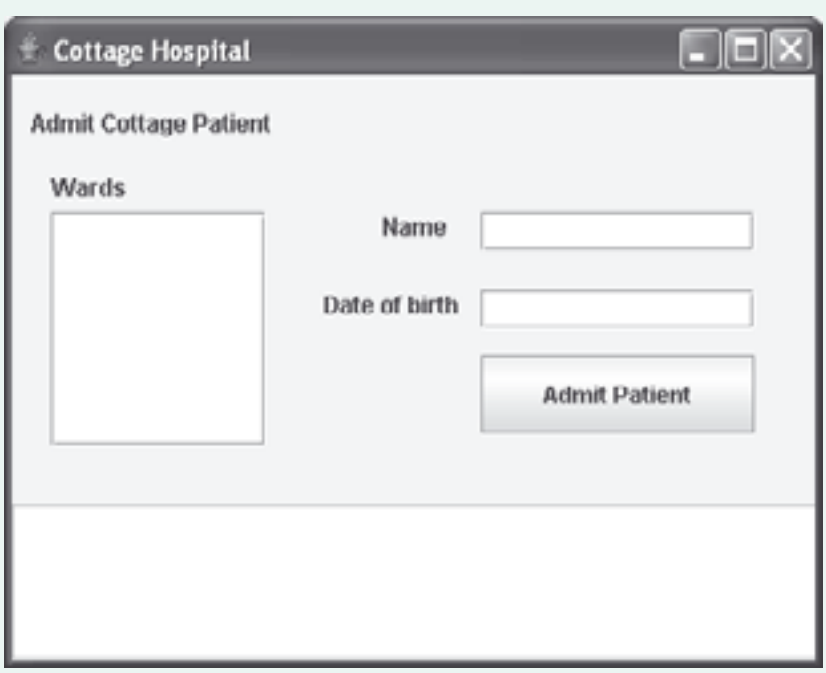


Figure 19 The incomplete GUI

(b) **Carry out step B4** (Write code to obtain a reference to the coordinating object.)

First open the source code for the `AdmitPatientScreen` class in the package `gui`, in the Source Editor.

Look back at part (e) of Exercise 2. You will need to add the same two import statements to the code; in addition, the package `m256date` must be imported. That is, add the following statements just before the class declaration:

```
import cottagehospcore.*;
import java.util.*;
import m256date.*;
```

Now, using part (e) of Exercise 2 as a guide, add the code making the coordinating object accessible to the GUI class.

(c) **Carry out step B5** (Write code to initialise components.)

Step (e) of Exercise 2 also shows how to code the constructor to initialise `wardList` with the collection of `Ward` objects. Do this now.

To initialise `outcomeArea` with the text `Outcome:`, add the following as the final statement in the constructor.

```
outcomeArea.setText("Outcome:");
```

Run the project to test what you have done so far. The names of the wards should now be displayed, and the text `Outcome:` should be displayed in the output area.

(d) **Carry out step B6** (Add events to the widgets, and write the event handling code.)

In the Form Editor select the `Admit Patient` button, right-click on it and choose `Events|Action|actionPerformed`.

You will automatically be taken to the method `admitPatientButtonActionPerformed()` in the source code, and here you will need to write code to handle the event.

Your code should first gather the values (patient name and date of birth, and the selected ward) that the user has supplied.

To do this, use the `getText()` method to obtain the name from `nameField`, assigning the result to a `String` variable `theName`. Here is the code needed for this.

```
String theName = nameField.getText();
```

Write a similar statement to get the date from `dobField` and assign it to another `String` variable `theDOB`.

At this point the date of birth is represented as a `String` object and you need to construct an `M256Date` object from this. The `M256Date` constructor will throw an exception if the string supplied is not a valid date format (that is, if it is not of the form dd/mm/yy or dd/mm/yyyy) and so it must be called from inside a `try` block. In the `catch` block, as shown below, the user is informed of a problem with the date, by the use of the `setText(aString)` method to set the text in the output area. In this case, execution of the method should then stop.

Now add the code below.

```
M256Date theDate = null;
try
{
    theDate = new M256Date(theDOB);
}
catch (Exception e)
{
    outcomeArea.setText("Error: invalid date.");
    return;
}
```

Next use `getSelectedValue()` to get the selected ward from `wardList`, cast the object returned to `Ward`, and assign it to a `Ward` variable `theWard` (as in step (f) of Exercise 2).

Now you have all the arguments ready, so write code to send a coordinating message `admit(theName, theDate, theWard)` to the coordinating object.

You are not quite finished! The user must receive feedback. So a final statement is needed which sets the text in `outcomeArea` to read `"Outcome: Patient X has been admitted to Y ward."` where X is the patient name and Y the ward name. Remember the `setText(aString)` method will be needed to provide this feedback.

When you have added this code, run the project and try the GUI out.

Note that this simple system does not display **persistence**; that is, if you admit a patient, when you exit the system the information about the new patient is lost.

Discussion

Here is an extract of the source code for the GUI. You can find a complete solution in the project `Cottage_Hospital_Ex_5_Sol`.

Imports (step B4):

```
import cottagehospcore.*;
import java.util.*;
import m256date.*;
```

Instance variable declaration (step B4):

```
private CottageHospCoord cottageHospCoord;
```

Constructor (steps B4 and B5):

```
public AdmitPatientScreen()
{
    initComponents();
    cottageHospCoord = new CottageHospCoord();
    Collection<Ward> wards = cottageHospCoord.getWards();
    wardList.setListData(wards);
    wardList.setSelectedIndex(0);
    outcomeArea.setText("Outcome:");
}
```

Handling code (step B6):

```
private void admitPatientButtonActionPerformed(java.awt.event.ActionEvent evt)
{
    String theName = nameField.getText();
    String theDOB = dobField.getText();
    M256Date theDate = null;
    try
    {
        theDate = new M256Date(theDOB);
    }
    catch (Exception e)
    {
        outcomeArea.setText("Error: invalid date.");
        return;
    }
    Ward theWard = (Ward)wardList.getSelectedValue();
    cottageHospCoord.admit(theName, theDate, theWard);
    outcomeArea.setText("Outcome: Patient " + theName +
                        " has been admitted to "+ theWard + " ward.");
}
```

SAQ 1

Here is the start of a sequence diagram, similar to the one given in Figure 5, for the interaction that takes place in the event handling code above. What are the messages corresponding to the labels (a), (b) and (c) respectively?

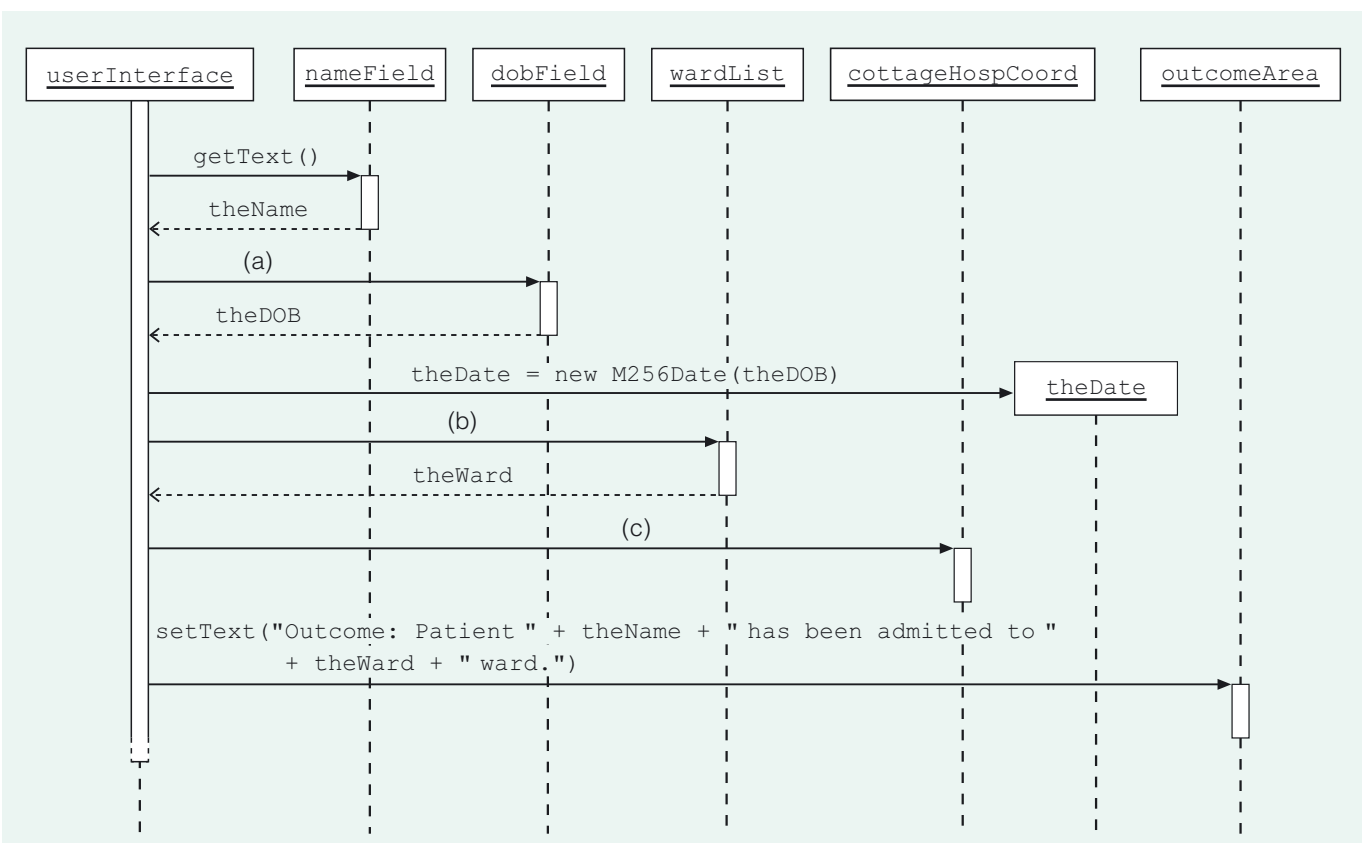


Figure 20 An incomplete sequence diagram

ANSWER.....

(a) `getText()`

(b) `getSelectedValue()`

(c) `admit(theName, theDate, theWard)`

3.3 Validating input

The handling code you developed in Exercise 5 checks only the date part of the input data, and only in a very basic way (checking that it is of the form dd/mm/yy or dd/mm/yyyy). In a more sophisticated GUI, you would probably implement many other checks, ensuring for instance that the date of birth supplied is not in the future and that the name is a valid name.

In reality, then, the GUI would need to be extended so that, before it initiates the use case, it checks that the data is valid and complete. You would then need to decide what should happen if the data is invalid or incomplete. A sensible course of action is to advise the user that there is a problem and give the details, as we have already done in the case of a date of the wrong form. In a real project the client would need to be consulted on what course of action should be adopted.

For simplicity, here we will just add code to check that the name is not empty. Once a patient has been admitted, their name and date of birth should be cleared (otherwise there is a danger that the user could make an error and admit them twice). So we will also add code which sets the text in both `nameField` and `dobField` back to the empty string.

While the handling code is being modified we will also take the opportunity to *refactor* it; that is, to amend its structure whilst retaining the overall behaviour. Refactoring will prevent the handling code from becoming overly long and will help the high-level logic to be more easily understood. It would be better if the handling code simply called, as follows, on **helper methods**, `doAdmit()` and `resetFields()`, to take care of all the details:

You met the concept of refactoring in *Unit 9*, Section 4, in the context of the design of a system.

A helper method exists just to assist other methods in the same class.

```
private void admitPatientButtonActionPerformed (java.awt.event.ActionEvent evt)
{
    doAdmit();
    resetFields();
}
```

As well as performing the steps which in Exercise 5 were found in the handling code, `doAdmit()` will validate the input. The steps that it must carry out can be described as follows.

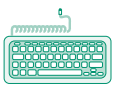
This type of description, which has some of the structure of code but without the syntax of a programming language, is sometimes called **structured English**.

1 Obtain the name
2 If the name is empty
    display an appropriate message in the output area and return from the method
3 Obtain the date string
4 Attempt to create an `M256Date` object
5 If an exception is thrown
    display an appropriate message in the output area and return from the method
6 Admit the patient
7 Display an appropriate message in the output area

The helper method `resetFields()` will simply set the text in `nameField` and `dobField` to the empty string as follows.

```
private void resetFields()
{
    nameField.setText("");
    dobField.setText("");
}
```

In the next exercise you will complete the implementation of the changes described above.



Exercise 6

In NetBeans open the project `Cottage_Hospital_Ex_6`, which is in the folder `M256\M256Code\Exercises\Unit13`.

The `AdmitPatientScreen` class in the package `gui` contains a partial implementation of the modifications described above. Open the source code for this class in the Source Editor.

- The planned refactoring has been done, i.e the handling code has been streamlined, and the helper methods have been added. You will find them immediately after the event handling method `admitPatientButtonActionPerformed`.
- The method `resetFields()` is complete.
- However, the body of the method `doAdmit()` is incomplete.

Complete the `doAdmit()` method by following the description above and studying the handling code used previously in Exercise 5.

When you have done this, run the project and confirm that if no name is supplied, or if the date of birth supplied is not in the form dd/mm/yy or dd/mm/yyyy, the user receives an appropriate warning.

Discussion

Our code is as follows. You can find this in the project `Cottage_Hospital_Ex_6_Sol`.

```
private void doAdmit()
{
    String theName = nameField.getText();
    if (theName.equals(""))
    {
        outcomeArea.setText("Error: You need to enter a name.");
        return;
    }
    String dateString = dobField.getText();
    M256Date theDate = null;
    try
    {
        theDate = new M256Date(dateString);
    }
    catch (Exception e)
    {
        outcomeArea.setText("Error: invalid date.");
        return;
    }
```

```
        Ward theWard = (Ward)wardList.getSelectedValue();
        cottageHospCoord.admit(theName, theDate, theWard);
        outcomeArea.setText("Outcome: Patient " + theName +
                            " has been admitted to "+ theWard + " ward.");
    }
```

In this section you have learnt about a systematic approach to building a GUI, involving the steps needed to plan it and those needed to implement it. You then applied this approach to the process of building a GUI for the *Admit Cottage Patient* use case of the Cottage Hospital System. You also learnt how to extend this interface to deal with invalid input data.

4 Combining use cases into a single interface

In Sections 2 and 3 you constructed GUIs for each of the two Cottage Hospital System use cases as two separate programs. Obviously this would not be very convenient in practice: a way is needed to allow the user to switch between screens within the same program. There are various techniques for doing this, but the solution our imaginary GUI designer has specified is to use tabbed screens.

In this section you will learn:

- how tabbed screens can be used to implement a GUI;
- how to coordinate the operations of tabbed screens.

Swing contains a class `JTabbedPane` for implementing tabbed screens. Like a `JScrollPane`, a `JTabbedPane` is designed to be a container for other components; a single `JTabbedPane` is typically the container for several tabbed screens.

A `JTabbedPane` is normally used by adding panels (`JPanel` objects) to it, each of which implements a separate (tabbed) screen. Each panel that is added 'hides' the previous ones, except for the protruding tab. Thus, the user can move from one panel to another by clicking the relevant tab.

In the following exercise, you will investigate a partially complete implementation of the Cottage Hospital System GUI, which has been structured as two tabbed screens as discussed above. You will then complete the implementation by implementing the necessary connections between the two screens. The exercise leads you step by step through what is required.

Exercise 7

In NetBeans open the project `Cottage_Hospital_Ex_7`, which is in the folder `M256\M256Code\Exercises\Unit13`.

This contains an implementation of the Cottage Hospital System GUI as described above.

(a) Run the project. This will reveal a GUI containing two tabbed screens – you can bring up the screen for either use case by clicking on the corresponding tab.

Close the GUI.

(b) Now open the source code for the `CottageHospitalGUI` class in the package `gui`. If necessary, click on the `Design` button to switch to the Form Editor and expand the tree in the `Inspector` window to see the structure of the GUI implementation.

We have packaged up the implementations (from Exercises 2 and 6) of the `List Cottage Ward's Patients` and `Admit Cottage Patient` screens. For each screen, instead of the components (scroll panes, labels etc.) being placed directly on a `JFrame`, they are now nested in one of two `JPanel`s (referenced by `listWardsPatientsScreen` and `admitPatientScreen` respectively).

The two `JPanel`s are nested inside a single `JTabbedPane`, `jTabbedPane1`, which in turn is nested inside the overall `JFrame`.

To distinguish between the ward lists in the two screens, we have renamed them `wardList1` (on the `List Cottage Ward's Patients` screen) and `wardList2` (on the `Admit Cottage Patient` screen).

Implementing each screen as a separate `JPanel` is a great improvement on the technique of adding widgets directly to a `JFrame`. Not only can a number of screens now be combined in a single GUI, but the screens become reusable. For example, suppose the designer decided not to use tabbed screens after all, but to navigate between screens in some other way. Instead of having to start again from scratch, the implementer would simply incorporate the existing panels that had already been written into the new structure, so that much of the existing work could be reused unchanged.

(c) Run the project and click on the `List Patients` tab. Click on `Heron` to display the details of the three patients there.

Now click on the `Admit Patient` tab and admit a new patient Claudius Ptolemy, born 01/01/46, to Heron ward (it must be Heron: make sure you do not admit the new patient to any other ward by mistake). In the output area the message

```
Outcome: Patient Claudius Ptolemy has been admitted to Heron ward.
```

should appear.

Click on the `List Patients` tab again. The list of patients on `Heron` ward should still be visible – but no Claudius Ptolemy!

This is because the code does not arrange for the data in the list of patient details to be reloaded, so it is still showing the same list as before. To make Claudius appear you need to reselect `Heron` ward by clicking on another ward, and then on `Heron` again. This is unlikely to be acceptable to users, so we need a way to make sure that the `List Cottage Ward's Patients` screen always shows up-to-date information.

There is another issue to consider. Click the `Admit Patient` tab again. When the screen reopens, the message

```
Outcome: Patient Claudius Ptolemy has been admitted to Heron ward.
```

is still displayed. This is also undesirable: it would be preferable if this old message were cleared.

The simplest approach is for each screen to update automatically whenever it is opened. Java provides a special type of event to cater for this need – the `Component` event. The particular component event `componentShown` enables a screen to be updated when it opens.

Close the GUI.

(d) The existing project already includes code to display a list of the patients on the selected ward. This code is used by the handler method `wardList1ValueChanged(evt)`. Now we want to use the same code in a different handler method, so rather than duplicate it we have factored it out as a helper method.

The new method can be found in the source code for `CottageHospitalGUI`, immediately after the `resetFields()` and `doAdmit()` helper methods added previously. The method is as follows.

```
private void displayPatients()
{
    Ward theWard = (Ward)wardList1.getSelectedValue();
    Collection<Patient> patients = cottageHospCoord.getPatients(theWard);
    List<String> patientData = new ArrayList<String>();
    for (Patient eachPatient : patients)
    {
        patientData.add(eachPatient.getName() + ", " + eachPatient.getAge());
    }
    patientList.setListData(patientData);
}
```

To make the List Cottage Ward's Patients screen reload the patient data every time it is opened, all that is required is to add a handler method for componentShown events from this source, and then add a statement to the handler code that calls displayPatients().

In the Inspector window for CottageHospitalGUI, expand the component tree and right-click on listWardsPatientsScreen. Choose Events|Component|componentShown. When you are taken to the handler method, insert the following statement.

```
displayPatients();
```

Run the project and repeat the first experiment of part (c) above. You should now find that patient data is reloaded when the List Cottage Ward's Patients screen is opened.

(e) The project also includes another new helper method resetOutcomeMessage(), which sets the text of the output area back to "Outcome:".

By following the same procedure as in part (d) above, add a handler method which will respond to componentShown events from the Admit Cottage Patient screen. The handler code should call the resetOutcomeMessage() method, thus resetting the text in the output area.

Run the project again, and check that now when you admit a patient, then leave the Admit Cottage Patient screen and return to it again, the message in the output area is correctly reset.

Discussion

There is no extended discussion for this exercise. However, you can compare what you have done with our answer, which is in the project Cottage_Hospital_Ex_7_Sol.

The simple approach to updating that has been implemented in Exercise 7 certainly works, but it is rather inefficient. It reloads the data in the List Cottage Ward's Patients screen every time the screen is opened, irrespective of whether anything has actually changed. In a real system this might not be appropriate. There are more sophisticated ways of ensuring that the components of an interface are kept up to date when the core system changes state. These will be discussed briefly in *Unit 14*.

In this section, you have learnt how to use tabbed screens to combine the separate screens for different use cases into the same GUI. You have also learnt how these tabbed screens are kept in step with each other.

You now have a fully functioning GUI for the Cottage Hospital System. Although this is a very simple GUI, its design and implementation illustrate many important features and techniques that can be used for much more complex systems.

5 The GUI for the Hospital System

In the previous sections of this unit you were introduced to a structured approach to building GUIs, and you also learnt how to use NetBeans' GUI designer facilities to implement a GUI for a simple system involving two use cases.

In this section you will apply these skills to completing an implementation of the Hospital System GUI. The core system we will use is as you encountered it at the end of *Unit 10*: six use cases have been implemented. In the exercises that you will work through in this section, the GUI screens for five of them are complete; the screen for the *Treat Patient* use case is nearly complete but some gaps remain. You will complete the implementation of this screen by adding some short pieces of code.

Although, of course, the Hospital System GUI is more complex, its overall structure resembles that of the GUI constructed for the Cottage Hospital System. As you work through this section you should recognise many of the ideas you encountered earlier.

5.1 The structure of the Hospital System GUI

Use cases

The six use cases implemented in the Hospital System core system are as follows.

Admit Patient

Treat Patient

Discharge Patient

List Ward's Patients

List Team's Patients

List Patient's Treatment

In this section you will only need to refer to the *Treat Patient* use case, whose description is as follows.

> The administrator identifies the patient and the doctor; the doctor must be in the team that cares for the patient. The system records that the doctor has treated the patient.

Structure

In *Unit 12* we decided that users would navigate between the screens of the Hospital System GUI by using tabbed screens. This is similar to the Cottage Hospital System GUI. However, in the Hospital System GUI all six of the use case screens have in common:

- an output text area for displaying the outcome of the use case;
- an `Exit` button.

In our implementation the output area and the `Exit` button have been made common to all the screens, as shown in Figure 21.

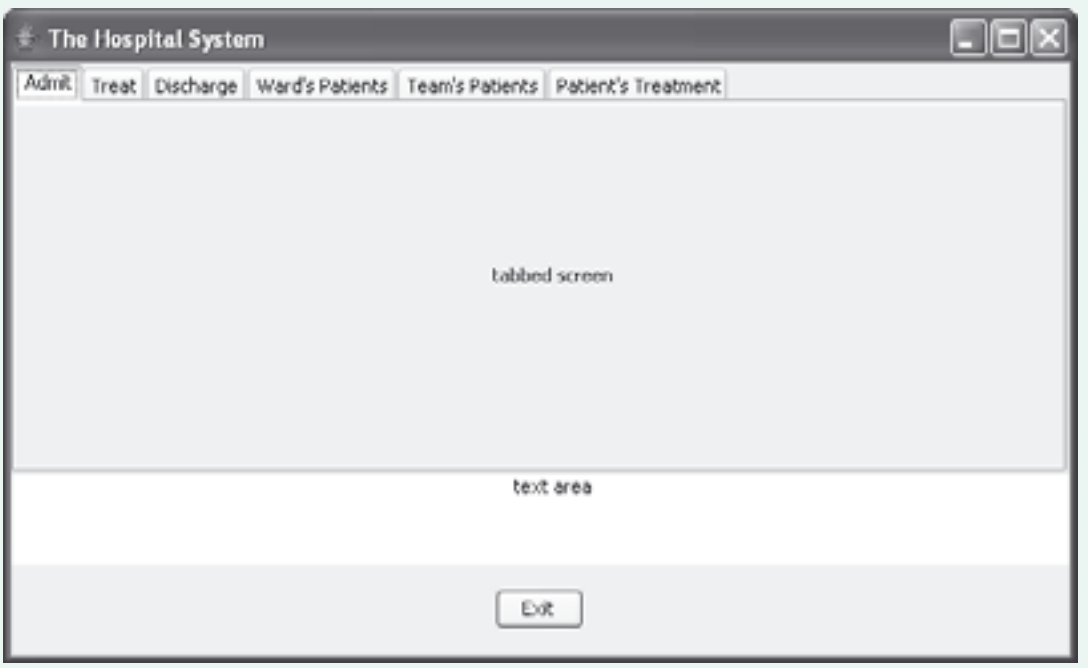


Figure 21 Tabbed screens above a shared text area and `Exit` button

The upper part of the GUI is implemented by a `JTabbedPane` containing several `JPanel`s, each corresponding to a tabbed screen for a use case. The bottom part of the window is occupied by the single text area and the `Exit` button, which the use cases all share.

The overall structure of the GUI is outlined in Figure 22. For clarity it does not show the components (lists, buttons etc.) contained in the panels corresponding to each use case (`admitPatientPanel`, `treatPatientPanel` etc.). The panel `sharedPanel` houses the shared part of the GUI.

Note that in contrast to the Cottage Hospital System GUI, which used only the null layout, the Hospital System GUI uses two layouts: border layout and grid bag layout. These are described in the Appendix, but to understand Figure 22 all you need to know is that in a border layout North lies above Center, which lies above South, and a grid bag layout arranges components in a grid a bit like a spreadsheet.

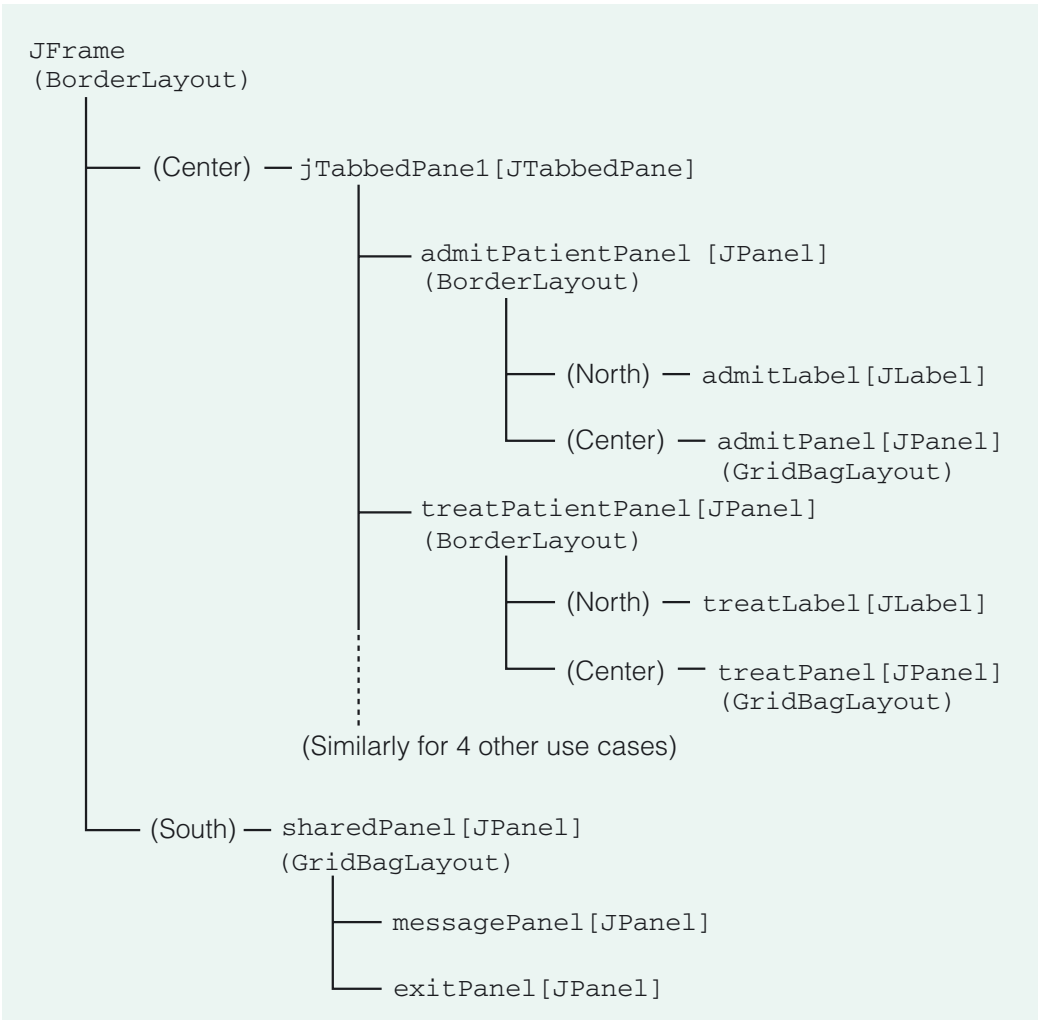


Figure 22 The structure of the GUI

5.2 The coordinating object

Recall that to create a coordinating object in the Cottage Hospital System, we used the `new` operator as follows.

```
CottageHospCoord cottageHospCoord = new CottageHospCoord();
```

In the Hospital System, however, creating a coordinating object is slightly different since there is no public constructor for the coordinating class. The Hospital core system instead offers clients a static method, `getHospital()`, which returns a coordinating object. As you will learn in *Unit 14* (you do not need to know the details), this enables the Hospital System to exhibit *persistence*: that is, the changes the user makes to the system's state are not discarded when the user exits from the system, but are saved and preserved when the system is run again.

Here is the Javadoc description for this static method.

getHospital

`public static HospCoord` **`getHospital`**`()`

Creates and returns a new `HospCoord` object. Reads in the state of the object from the file `Hospital.data`; if there is no such file, returns the object in its initial state.

Returns:

a new `HospCoord` object

Hence, the GUI obtains a reference to the coordinating object via the following statement, where `hospital` is an instance variable of type `HospCoord`.

```
hospital = HospCoord.getHospital();
```

As you may have guessed, saving the current state of the system is the responsibility of another of the `HospCoord` methods. Here is its Javadoc description.

Neither `getHospital()` nor `save()` is documented in the `HospCoord` class description in the implementation model. Recall that in Section 7 of *Unit 10*, we said that because of the artificial simplicity of the Hospital System, some methods had to be added to those in the implementation model to enable it to function as required.

save

`public void` **`save`**`()`

Saves the state of the receiver to the file `Hospital.data`.

When the user exits the GUI, a `save()` message should be sent to the coordinating object, which will respond by saving the system state to a data file.

SAQ 2

Given that an event is generated when a window is closed, how could this be used to send the `save()` message?

ANSWER.....

The event could have an associated handler method which causes the message to be sent.

The answer to SAQ 2 describes exactly what is done in the Hospital System GUI. The `formWindowClosing(evt)` event handler causes the `save()` message to be sent to the coordinating object.

5.3 The *Treat Patient* screen

In this section you will examine the *Treat Patient* screen, and then you will have an opportunity to put what you have learnt in previous sections into practice by completing the implementation of this screen.

Treat Patient was one of the use cases for which we produced a GUI design in *Unit 12*. The design sketch, from Figure 15 of that unit, appears below.

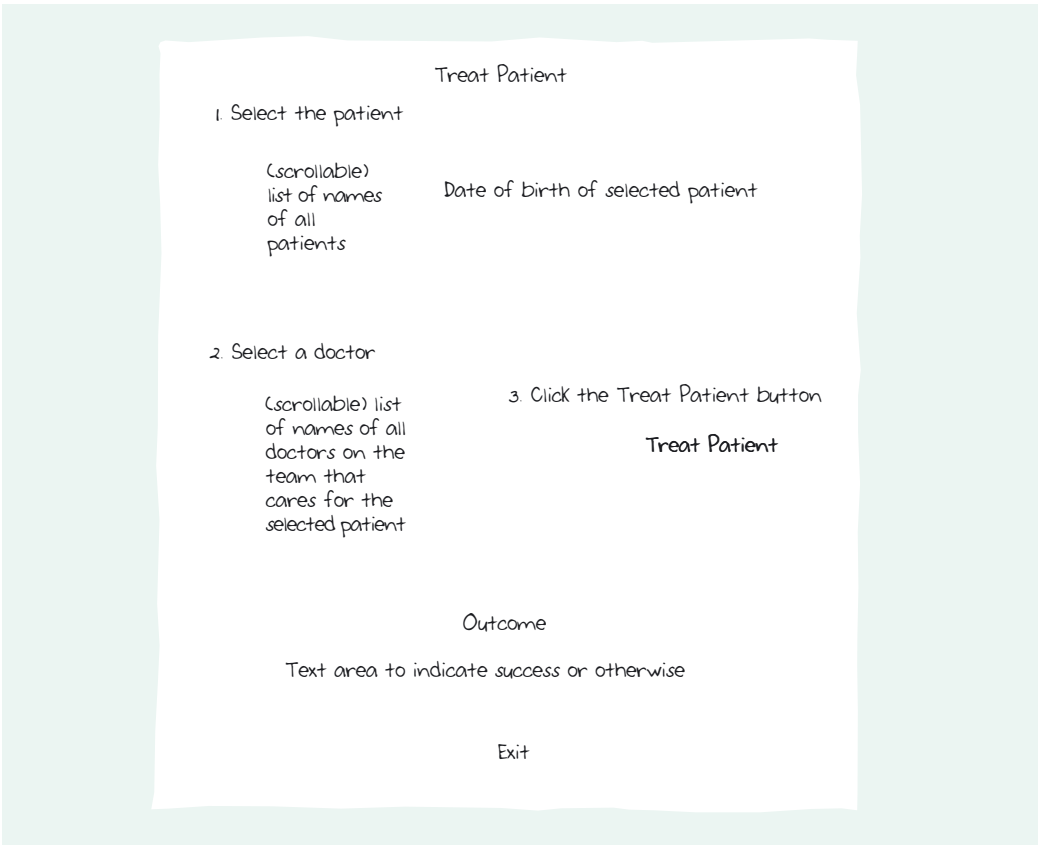


Figure 23 The screen design for the *Treat Patient* use case

In the implementation of this design using Swing components, the lower portion of the window is implemented by a `JPanel`, `sharedPanel`, which houses the output area and **`Exit`** button, shared by all the GUI screens.

The upper portion of the window is a `JPanel` placed on a `JTabbedPane`. The `JPanel` holds the **`Treat Patient`** screen, and is further subdivided. The patient details appear at the top, and a `JPanel` nested lower down holds the components related to recording the treatment.

The GUI design is repeated in Figure 24, with the `JPanel`s represented by dotted rectangles. We have also indicated the names of key variables (such as `patientList1` and `dobField1`) to which messages will be sent. For components which will not be message receivers, such as the `JLabel` and `JScrollBar` objects, we have omitted this information.

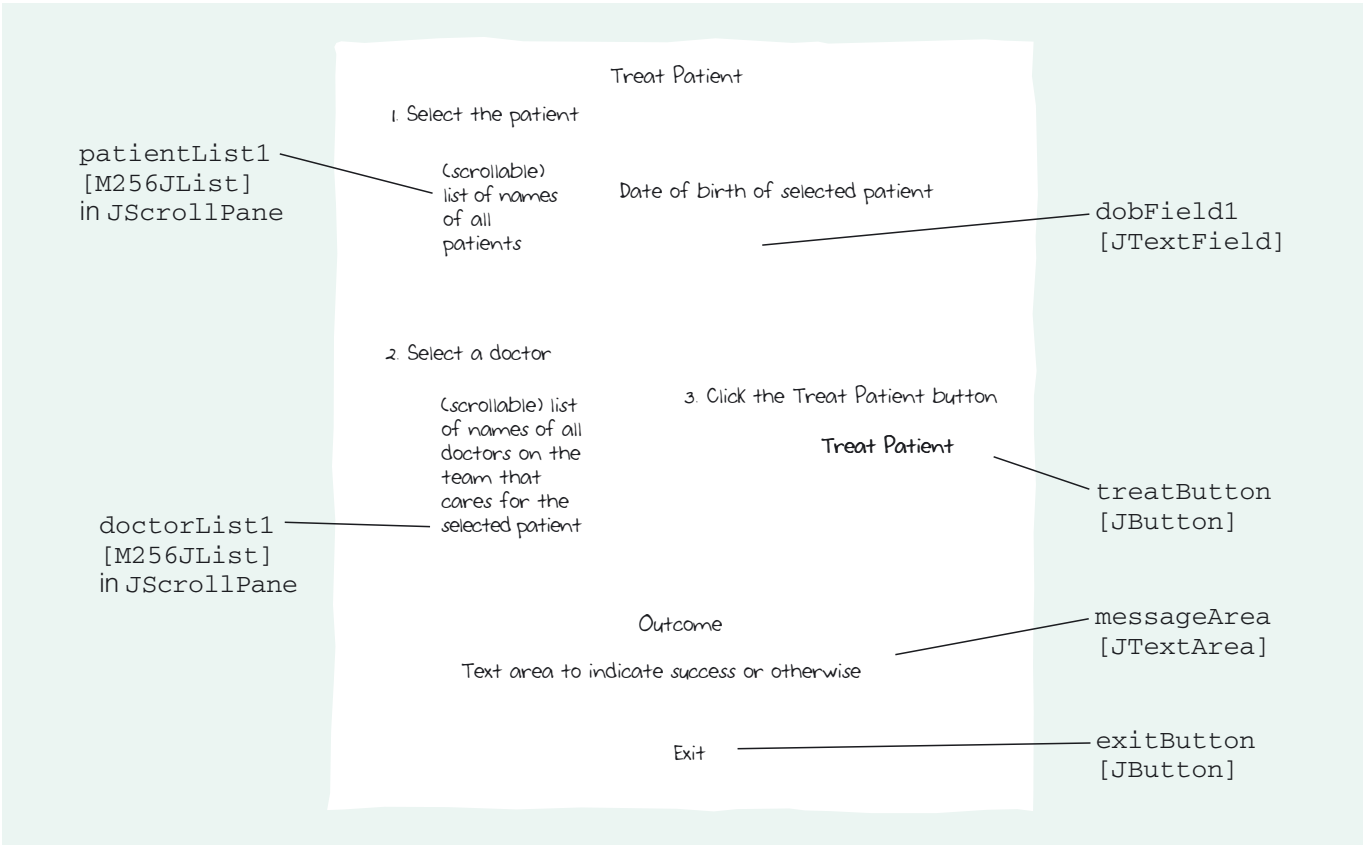


Figure 24 How the GUI design is realised with Swing components

This provides an overview of how the screen is structured. The next step is to examine the corresponding code.

Exercise 8

In NetBeans open the project `Hospital_Ex_8`, which is in the folder `M256\M256Code\Exercises\Unit13`. (Note that this project will also be used for Exercise 9.)

This project contains three packages:

- `hospitalcore` is the Hospital System core system package, as you saw in *Unit 10*;
- `hospitalgui` contains a partial implementation of a GUI interface to the core system;
- `m256people` is the Hospital System's people-related utilities package, as you saw in *Unit 10*.

`hospitalgui` also contains several other classes (`DisplayTableCell Renderer` etc.), whose purpose is to set the format that a `JList` will use to display objects. You are not expected to examine these classes: you will not need to work with them directly.

Open the `HospGUI` class in the package `hospitalgui`, in the Source Editor.

This is a purposely incomplete version of the Hospital System GUI which has been created for this exercise and the next one. Since the code is so long, we have folded up most of it, allowing you to concentrate better on the section concerned with the `Treat Patient` screen, which is implemented using a `JPanel`, `treatPatientPanel`.

(a) Run the project. It should open showing the `Admit Patient` screen. Open the other screens in turn by clicking on the relevant tabs and check that they look as you expect. Note that, as there are no patients in the hospital, parts of the `Treat Patient`, `Discharge Patient` and `List Patient's Treatment` screens are not shown.

Close the GUI.

Now look at the source code for the class `HospGUI` in the package `hospitalgui`, in the Source Editor. Where is the coordinating object created?

Locate the methods relating to the `Treat Patient` screen. This section of the GUI code begins as follows.

```
// ************************************************************
// The methods in this section are concerned with the Treat Patient screen
```

The instance variable `patient` is used in these methods. It references the last `Patient` object selected, or the new `Patient` object if a patient has just been admitted. When a screen containing a list of patients is opened, the `patient` variable is used to set the list selection.

(b) What is the purpose of the method `openTreat()`? What event on what component should invoke this method?

(c) What is the purpose of the method `patientChanged()`? What event on what component should invoke this method? Which statement in `patientChanged()` is responsible for ensuring that the right doctors are displayed?

(d) What is the purpose of the method `recordTreatment()`? (Note that some of the code for this method is missing; you will be asked to supply it in the next exercise.) What event on what component should invoke this method?

Discussion.....

(a) The coordinating object is created in the constructor, `HospGUI()`, via the code `hospital = HospCoord.getHospital();`

(b) The purpose of `openTreat()` is to ensure that, in the list of patients on the **`Treat Patient`** screen, the one that is shown as selected is the one referenced by the `patient` instance variable and that the output area is cleared of any message. (Alternatively, if there are no patients, then this information is displayed.) This method should be invoked whenever the **`Treat Patient`** screen becomes visible. Thus it should be invoked by the `Component` event `componentShown` on the `treatPatientPanel` component.

(c) The purpose of `patientChanged()` is to ensure that when the user selects a different patient from the list of patients, or when the list of patients is reset:

(i) the `patient` instance variable is updated to reflect the selected patient;

(ii) the date of birth of the patient is displayed;

(iii) the names of the doctors on the team that cares for the patient are displayed so that the user can select an appropriate doctor.

This method should be invoked by the `ListSelection` event `valueChanged` on the `patientList1` component.

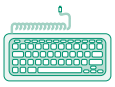
The statement

```
doctorList1.setListData(hospital.getDoctors(hospital.getTeam(patient)));
```

is the one that ensures that the right doctors are displayed. It gets the team for the current patient, and then gets and displays the collection of doctors who are on that team.

(d) The purpose of `recordTreatment()` is to ensure that when the user clicks on the **`Treat Patient`** button, the core system records the treatment of the selected patient by the selected doctor and a suitable message appears in the output area. This method should be invoked by the `Action` event `actionPerformed` on the `treatButton` component.

In the next exercise you will complete the implementation of the *Treat Patient* screen.



Exercise 9

In NetBeans run the project `Hospital_Ex_8` which you examined in the previous exercise.

(a) From the `Admit Patient` screen, do what is necessary to admit the patient Ms Jenny White, born 21/11/67, in the care of the team Orthopaedics A. To which ward is she admitted?

(b) Inspect the other screens and check that they now display what you expect. In particular note the names of the doctors that appear on the `Treat Patient` screen. To which team do you think these doctors belong?

(c) You are now going to try to record the treatment of Ms Jenny White by a doctor. Since the code for this part of the interface is incomplete, for now the treatment will not actually get recorded, and this experiment is just to find out exactly what does happen.

In the `Treat Patient` screen, ensure that the patient Ms Jenny White and the doctor Mr Edward Grundy are selected and then click the `Treat Patient` button. What happens? Check the `List Patient's Treatment` screen as well.

Close the GUI.

(d) Now open the source code for the class `HospGUI` in the package `hospitalgui`, in the Source Editor, and add the necessary statements to the code for the `recordTreatment()` method. You should need to add only two statements. The first sends a message to the coordinating object. The second is responsible for seeing that after the treatment has been recorded the output area will show a message such as the following.

```
Treatment of Ms Jenny White by Mr Edward Grundy has been recorded.
```

Note that the variable which references the `JTextArea` that implements the output area is `messageArea`.

Run the project again. You should find that Ms Jenny White is still a patient in the hospital, because the state was saved from the previous run.

Once again, do what is necessary to record the treatment of Ms Jenny White by Mr Edward Grundy. What happens this time? Remember to look at the `List Patient's Treatment` screen. What do you find?

Close the GUI.

(e) You can discover what needs to be done if you check the events of the `Treat Patient` button. To do this, select `treatButton` in the `Inspector` window, then right-click and select `Properties`. Clicking on the `Events` button at the top reveals which events have associated handler methods.

Make the necessary additions to implement the relevant event handler, then run the project again. This time you should be able to record the treatment of Ms Jenny White.

Discussion

You can find a complete solution in the project `Hospital_Ex_8_Sol`.

(a) You should have entered `Ms` for the title, `Jenny` for the first name, `White` for the surname and `21/11/67` for the date of birth. Check that `female` has been selected and then click on `Orthopaedics A` to select the team. Finally click the `Admit Patient` button. When you are asked to confirm the details, click `OK`.

The message in the output area should tell you that the patient has been admitted to the ward `Lower Loxley`.

(b) In the `Treat Patient` screen the names of the doctors Mr David Archer, Mr Edward Grundy and Ms Brenda Tucker appear. Since the selected patient is

Ms Jenny White, these doctors ought to be in the team Orthopaedics A, the team that cares for Ms Jenny White.

(c) When you click on the **Treat Patient** button nothing happens (apart from the button looking as if it is pressed and then released). The **List Patient's Treatment** screen claims that no doctors have treated Ms Jenny White.

(d) You need to add the following code at the end of the `recordTreatment()` method.

```
hospital.recordTreatment(patient, doctor);
messageArea.setText("Treatment of " + patient.getName()
                    + " by " + doctor.getName() + " has been recorded");
```

However, this makes absolutely no difference to the result. Nothing happens.

(e) No event handler methods have been added to the **Treat Patient** button. So nothing happens when it is clicked.

You need to add an `actionPerformed` event to the **Treat Patient** button, `treatButton`. To do so you will need to locate it in the **Inspector** window, or alternatively right-click on the button in the Form Editor.

You saw how to add an `Action` event to a button in Exercise 5.

In the `treatButtonActionPerformed()` method insert the following statement.

```
recordTreatment();
```

When you next run the project, you should find that clicking the **Treat Patient** button now has the desired effect. The output area shows the message required, and the **List Patient's Treatment** screen is as shown in Figure 25.

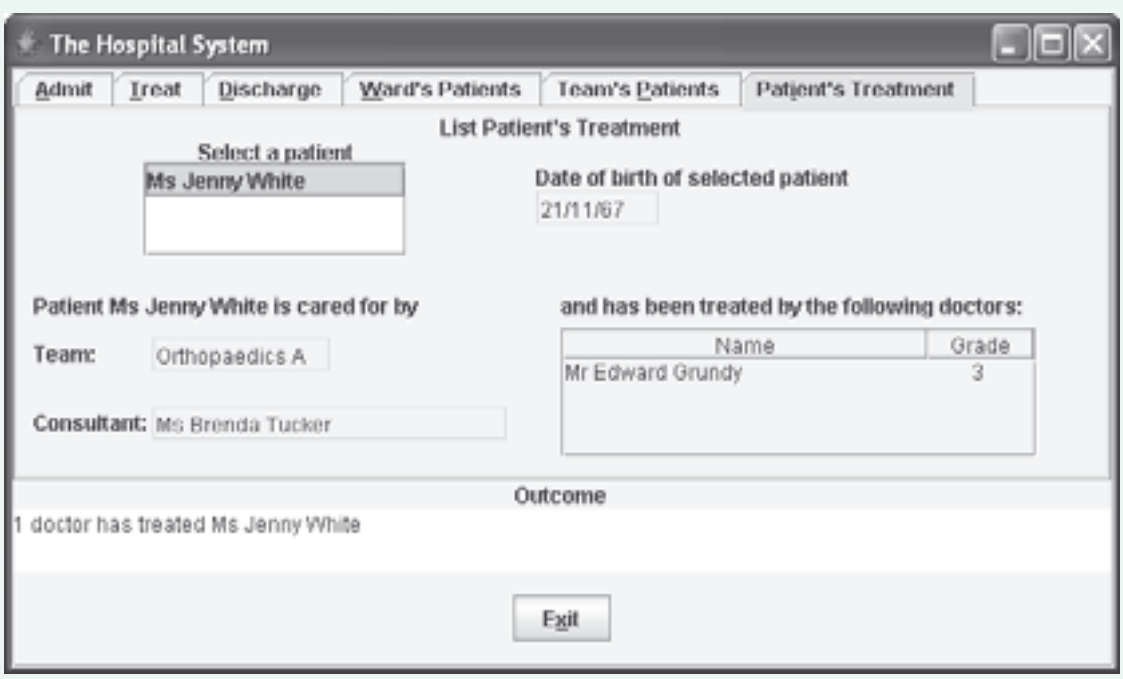


Figure 25 The **List Patient's Treatment** Screen

This completes our brief look at the GUI for the Hospital System. Although you have only examined a small part of it in detail, you should be able to appreciate that it is based on similar principles to the much simpler Cottage Hospital System GUI.

If you have time, you may want to investigate the code for some of the other Hospital System screens in more detail. But bear in mind that the code contains some features that go beyond anything you will be expected to do in M256.

6 Acceptance testing

With the construction of a user interface, the Hospital System is now complete, and in this short section we turn to checking whether it meets the user requirements, by carrying out *acceptance tests*.

You first met the idea of acceptance tests in *Unit 2,* Subsection 3.2.

In this section you will learn:

- more about what is meant by acceptance testing;
- how to specify acceptance test cases;
- how to perform an acceptance test based on a test case.

6.1 What is acceptance testing?

To check whether the requirements are met, acceptance testing is carried out. Acceptance tests are different from the tests you looked at in *Unit 10*. There, you carried out testing of the core system code, to check whether the code conformed to the design. The purpose of acceptance testing, on the other hand, is to see whether the system does what the user wants. It is quite possible that the system has been built correctly according to the design, but that the design itself was faulty and did not accurately represent the user requirements. Note that acceptance tests are not concerned with the usability of the GUI, but only with whether the system functions correctly. Usability testing is a separate issue.

As you learnt in *Unit 2*, the acceptance tests will usually be part of the contract between the client and the developers, and if the system fails the acceptance tests it will be rejected until the faults have been rectified. When the system passes the acceptance tests it is ready to be delivered to the client.

Acceptance testing is carried out by the client (or the client's agent). It is black box testing, which is to say that the client has no access to the internal working of the system and can judge it only from the external behaviour shown by the user interface. Indeed, the client is not concerned with how the system is implemented, but only with whether it performs as expected.

You have seen that testing code against a design often benefits from automation with JUnit or some similar framework. Frameworks for automating acceptance testing exist, although they are less widespread than for testing code. M256 will introduce you to only manual acceptance testing.

Testing is a complex area, and the following subsections are only an introduction to what can be involved in acceptance testing. However, they should equip you to carry out simple tests of your own, as well as serving as a good basis for further study of the topic.

6.2 Acceptance test cases

The requirements document for the Hospital System given in *Unit 2* lists one or more acceptance tests for each use case. We will begin by taking the first use case, *Admit Patient*, and showing how the test descriptions from the requirements document can be put into practice. Here is the description of the use case.

The administrator provides the system with the patient's name, sex and date of birth and identifies the team that cares for the patient.

The system identifies a ward of the appropriate type with empty beds; if there is a choice of such wards one of those with the largest number of empty beds is chosen.

If there is no appropriate ward with empty beds the administrator is informed of this fact.

If there is an appropriate ward with empty beds the system does the following two things:

1 It records:
   - the patient's name, sex and date of birth;
   - that the patient is under the care of the given team;
   - that the consultant that heads the team is responsible for the patient;
   - that the patient is on the ward.

2 It informs the administrator of the ward to which the patient has been admitted.

There are two acceptance tests for this use case, which we will label A1 and A2.

A1 Test that, when there is an appropriate ward available:
   - the admission of a patient is recorded correctly;
   - the administrator is informed of the ward to which the patient has been allocated.

A2 Test that, when no appropriate ward is available, an attempt to record the admission of a patient results in the administrator being informed that there is no appropriate ward available.

For each acceptance test, a *test case* is specified, just as you saw in *Unit 10*, Subsection 2.1, which documents:

- what the state of the system should be at the start of the test (the *fixture*);
- what input is to be provided;
- the expected result of the test, that is, the expected state of the relevant parts of the system after the test, along with any expected output data.

When the test is carried out, the fixture must be set up before the start, and the expected result checked at the end by looking at the appropriate GUI screens.

To specify a test case, you need to consult both the description of the use case and the description of the test.

The format used to specify acceptance test cases is illustrated by the example in Table 4, which we have written for test A1. We have invented some imaginary input data and an initial system state suitable for the test to be carried out. To record the information, it is useful to use a structured format such as this table, which is broadly similar to the tables used in recording test cases in *Unit 10*, Subsection 5.1.

The differences in the tables reflect their different contexts of use.

Table 4 Specifying an acceptance test case for *Admit Patient*

Use case: *Admit Patient* – **Test A1** Test admission when an appropriate ward is available	
Fixture Lower Loxley is a female ward and has 4 free beds. Grange Spinney is a male ward and has 7 free beds. Brookfield ward is a female ward and has 3 free beds. There are no patients under the care of Orthopaedics A. Ms Brenda Tucker heads Orthopaedics A. *Input* In the `Admit Patient` screen: Patient details Ms Fiona McPherson, 12/07/64 entered. `female` selected. `Orthopaedics A` selected. `Admit Patient` button pressed. Details confirmed.	*Expected result* The following will be recorded: The patient's name, sex and date of birth are Fiona McPherson, female and 12/07/64. The patient is under the care of Orthopaedics A. Ms Brenda Tucker is responsible for the patient. The patient is on Lower Loxley ward. The administrator will be informed that Ms Fiona McPherson has been admitted to Lower Loxley ward.

SAQ 3

Why is the patient expected to be admitted to Lower Loxley ward?

ANSWER.....

Because the use case description says:

> The system identifies a ward of the appropriate type with empty beds; if there is a choice of such wards one of those with the largest number of empty beds is chosen.

Since the patient is female, she must be admitted to a female ward. Given that there is a choice of female wards, the one with the highest number of empty beds should be chosen. This is Lower Loxley.

The initial state of the system was chosen deliberately so that there was more than one ward of the appropriate type, in order to test that the ward with the largest number of free beds was the one chosen.

There are two important points to notice about this test case.

Firstly, we have not described the state of the entire system, only the parts which are relevant to the use case, either because they influence the expected result or because they are expected to change as a result of the use case.

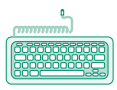
Secondly, the test case is specified in real-world terms, not in terms of software objects. So it says, for example, 'The patient is under the care of Orthopaedics A' and not 'The `Patient` object ... is linked to the `Team` object ...'. The acceptance test case defines tests which the client performs, and they are therefore couched in the client's own language.

Setting up fixtures

Before the client can run an acceptance test, the fixture must be created, that is, the system state must be set as specified for the start of the test. It would be possible to do this manually from the GUI, but another approach is to load the required system state from a special file. This is the solution we have adopted for the exercises which follow. In other words, when each project is run, the required system state will be set up automatically.

In the following exercise, you will perform the acceptance test corresponding to the test case A1 documented above.

Exercise 10



In NetBeans open and run the project `Hospital_Ex_10`, which is in the folder `M256\M256Code\Exercises\Unit13`.

In the `Admit Patient` screen do the following.

1 Provide the user inputs for test A1, as in Table 4:
   - enter the patient's details;
   - select `female` as the sex of the patient;
   - select `Orthopaedics A` as the team that cares for the patient.
2 Click on the `Admit Patient` button.
3 Confirm the details.
4 Check that the output is as expected.
5 Check, by inspecting the other screens, that the final state of the system is as expected.

Discussion

After you have entered the patient details and other input, clicked the `Admit Patient` button, and confirmed the details, the message

```
Patient Ms Fiona McPherson admitted to ward Lower Loxley
```

is displayed in the output area.

In the `List Ward's Patients` screen, if you select `Lower Loxley` you can see that Ms Fiona McPherson is recorded as being on Lower Loxley ward.

In the `List Team's Patients` screen, if you select `Orthopaedics A`, you can see that Ms Fiona McPherson is recorded as being cared for by Orthopaedics A.

In the `List Patient's Treatment` screen, if you select `Ms Fiona McPherson`, you can see that Ms Brenda Tucker is recorded as being responsible for the patient, and that the patient's date of birth is recorded as 12/07/64.

These actual results agree with the expected ones, and the system has therefore passed acceptance test A1.

Now consider acceptance test A2. The purpose of this test is to check that the system behaves correctly when no appropriate ward is available. What might be a suitable initial state for this test case? Obviously there are many different possibilities, but a simple approach is to use the same initial state as for test A1, but with the male ward, Grange Spinney, full. Then we can attempt to admit a male patient (see Table 5).

Table 5 Specifying a second acceptance test case for *Admit Patient*

Use case: *Admit Patient* – **Test A2** Test admission when an appropriate ward is available	
Fixture Lower Loxley is a female ward and has 4 free beds. Grange Spinney is a male ward and has no free beds. Brookfield ward is a female ward and has 3 free beds. There are no patients under the care of Orthopaedics A. Ms Brenda Tucker heads Orthopaedics A. *Input* In the `Admit Patient` screen: Patient details Mr Jose Delgado, 01/02/74 entered. `male` selected. `Orthopaedics A` selected. `Admit Patient` button pressed. Details confirmed.	*Expected result* No changes will have been recorded. The administrator will be informed that no appropriate ward is available.

Exercise 11

In NetBeans open and run the project `Hospital_Ex_11`, which is in the folder `M256\M256Code\Exercises\Unit13`.

In the `Admit Patient` screen do the following.

1 Provide the user inputs for test A2, as in Table 5:
   - enter the patient's details;
   - select `male` as the sex of the patient;
   - select `Orthopaedics A` as the team that cares for the patient.
2 Click on the `Admit Patient` button.
3 Confirm the details.
4 Check that the output is as expected.
5 Check that the final state of the system is as expected.

Discussion

The following message should be displayed in both a dialogue box and, at the end of the test, in the output area:

```
There is no bed in a type male ward available for patient Mr Jose Delgado
```

This is the correct output. But how do you know that the patient has not in fact been recorded as being admitted to a ward, despite the message. This is why step 5 is needed, since it confirms that the state of the system did not change when the attempt was made to admit Mr Jose Delgado.

From the `List Ward's Patients` screen you can confirm that Mr Jose Delgado is not recorded as being on any ward.

From the `List Team's Patients` screen you can confirm that Mr Jose Delgado is not recorded as being under the care of any team.

From the `List Patient's Treatment` screen you can confirm that Mr Jose Delgado is not listed as a patient.

These actual results agree with the expected ones, and the system has therefore passed acceptance test A2.

In the final two exercises you will practise what you have just learnt by specifying a test case for a particular use case, and then performing a corresponding acceptance test.

Exercise 12

Consider the second use case, *Treat Patient* (labelled B), of the Hospital System, given below with its corresponding acceptance test, which we have labelled B1 (there was just one acceptance test specified in the requirements document).

> The administrator identifies the patient and the doctor; the doctor must be in the team that cares for the patient. The system records that the doctor has treated the patient.

B1 Test that the treatment of a patient, by a doctor on the team that cares for the patient, is recorded correctly.

Draw up a test case for this acceptance test in a format similar to test A1 above. Make sure the condition that the doctor is in the team that cares for the patient is satisfied.

Discussion

Table 6 shows just one example of a test case. The important point is that the patient's team and the doctor's team should be the same.

Table 6 Specifying an acceptance test case for *Treat Patient*

Use case: *Treat Patient* – **Test B1** Test for the treatment of a patient by a doctor on the team that cares for the patient	
Fixture Ms Fiona McPherson is under the care of A and E. Ms Susan Carter is a junior doctor in the A and E team. *Input* In the `Treat Patient` screen: Ms Fiona McPherson selected. Ms Susan Carter selected. `Treat Patient` button pressed.	*Expected result* The system will record that Ms Fiona McPherson has been treated by Ms Susan Carter. The administrator will be informed that treatment of Ms Fiona McPherson by Ms Susan Carter has been recorded.

Exercise 13

In NetBeans open and run the project `Hospital_Ex_13`, which is in the folder `M256\M256Code\Exercises\Unit13`, and perform the acceptance test corresponding to the test case given in the discussion of Exercise 12 above.

Discussion

You should find that the system passes the test.

Note that it is not possible for the user to attempt to record treatment of a patient by a doctor from the wrong team, because the GUI constrains the user to select the doctor from a list of only the doctors in the team which cares for the patient.

6.3 Extending the acceptance tests

You could work your way through all the other use cases of the Hospital System in a similar way, taking the acceptance tests specified in the requirements document and writing a test case for each.

However it is not usually possible to specify a completely adequate set of acceptance tests as early as the initial requirements document. At that stage of a project, the clients can say only what the use cases should be, and provide tests which check the functionality – what each use case should do. Only the 'typical' scenario in which everything goes right, and a few predictable variations, are considered to start with.

In practice, a wide range of additional acceptance tests will be needed, over and above those set out in the initial requirements document. The project will be developed *iteratively*, involving a rolling series of review meetings between the clients and the developers. The clients will usually be shown prototypes of the working system, including the GUI. Variations to use cases will be identified, such as what should happen in cases of invalid data (for example, when a user inputs a date of birth that lies in the future). The detailed design of the GUI will be firmed up, and this will involve making another series of decisions.

Eventually the requirements will take their final form, and then the clients will draw up comprehensive acceptance tests to ensure that the system is fit for purpose before they accept it. The developers may have some involvement, but the emphasis will be on the users, and whether the product meets their needs.

SAQ 4

What is the purpose of acceptance tests?

ANSWER.....

To determine whether a software system meets the users' needs and is fit for the purpose for which it is intended.

SAQ 5

Why are the tests specified with the initial requirements document unlikely to be a comprehensive set of acceptance tests?

ANSWER.....

Because during an iterative development process requirements may be modified, refined and extended, and so additional tests will be needed.

In this section you have learnt more about what is meant by acceptance testing. You have seen how to specify a test case for a particular acceptance test of a use case, and how to perform an acceptance test based on a test case. We also explained why it is not usually possible to draw up a complete set of acceptance tests on the basis of the initial requirements specification.

7 Summary

This unit began by introducing a small-scale version of the Hospital System, the Cottage Hospital System, which has only two use cases. You saw, using a very simple textual interface, how a user interface for this system can communicate with its core system.

You then constructed your first graphical user interface, using the GUI designer facilities of NetBeans, and learnt that this way of building a GUI is an example of component-based development.

You learnt about a systematic, step-by-step process for implementing a GUI based on a screen design for a use case, and worked through an example of applying this process. You saw how to extend a GUI to validate user input.

Any realistic system will involve more than one use case, so the next stage was to examine how an interface can be built that allows the user to switch between separate screens for use cases. Once several screens are involved, it is necessary to arrange for them all to keep in step, so that if one use case changes the data in the system, other screens get updated appropriately.

The focus throughout was on the *ideas* that underpin the implementation of GUIs, rather than the technicalities of Java programming (our GUIs were deliberately simple) and much of what you learnt would still apply if another language were being used.

Having completed the Hospital System by finishing its GUI, the system needed testing using the acceptance tests from the requirements document produced in *Unit 2*. You saw how to write test cases for these acceptance tests, and then performed corresponding acceptance tests on the Hospital System.

LEARNING OUTCOMES

After studying this unit you should be able to:

- describe and use each of this unit's key terms (summarised in the Glossary);
- write code to implement communication between a user interface and a core system, and explain the interactions involved;
- use the GUI designer facilities in NetBeans to create GUIs consisting of Swing components;
- write code to handle events;
- decide where event handlers should be used;
- explain how the use of Swing components exemplifies component-based development;
- apply a systematic process to building a GUI;
- write code to validate user input in simple cases;
- explain how a `JTabbedPane` can be used to combine more than one use case screen;
- write code to keep screens in step when changes are made to the state of the system;
- explain, on the basis of code excerpts, how an existing GUI works;
- write test cases for simple acceptance tests given in a requirements document, and carry out the tests;
- explain the role of acceptance testing, and why in reality there would be tests beyond those listed with the initial requirements.

Appendix
The AWT and Swing menagerie

A Java GUI consists of visual components arranged in a container according to the rules of some particular layout manager. A component typically implements an on-screen widget, with which the user can interact, normally via the mouse and keyboard. User interaction is sensed by the Java system as events. Each event is associated with the particular widget or container where it originates, and the programmer can make the program respond to the event in a particular way by writing appropriate event handling code in an event handler method.

Components and containers

Components

In Java, most components are instances of classes in the package `javax.swing` and have names beginning with `J`. Such classes are referred to as Swing classes. The Swing classes are platform independent: that is, a GUI implemented in Swing should present a similar appearance whatever computer platform it is run on.

There are many different Swing components, but in M256 we use only a few of them, listed below.

Label A `JLabel` displays some text or an icon, which can be used to describe something – another widget, for example.

Text field A `JTextField` contains a single line of text, which can display output and, if the field is editable, accept input.

Text area A `JTextArea` is like a `JTextField` but can contain multiple lines of text.

(Action) button A `JButton` is used as a way of instructing the program to perform some action.

Radio button A `JRadioButton` is either on or off, and allows a user to select between mutually exclusive options. Radio buttons must belong to a `JRadioButtonGroup`, only one of which can be on at a time.

Check box A `JCheckBox` is either ticked or not ticked, and allows a user to switch a particular feature on or off; when clicked on, it switches from one state to the other.

List A `JList` displays a list of selectable items, and can report the current selection. The class `M256JList` that we use in this unit is a subclass of `JList`, tailored to make it slightly simpler to work with.

Other common widgets that you may have met but which are not used in this unit are implemented by components such as `JMenu`, `JMenuItem`, `JScrollBar` and `JComboBox`.

Containers

The containers we use in M256 are as follows.

Frame A `JFrame` is a top-level container; a window which can be used to house all the other components of a GUI.

Panel A `JPanel` is lower-level container, which can be used to group components that are to be manipulated as a unit in some way.

Scroll pane A `JScrollPane` is a container which is wrapped around another component, such as a list, which needs to be scrollable in case its contents outgrow the available display area.

Tabbed pane A `JTabbedPane` is a container which holds a number of overlaid components in the same space. A row of tabs, one for each component, allows the user to choose which component is displayed by clicking on the corresponding tab.

Properties, methods and events

You need to know something about three aspects of a component if you are to use it successfully. These are its properties, its methods and the events it can respond to.

Properties

The properties of a component are its attributes. For example, components such as text fields, text areas and buttons have a `text` property that determines what text is shown in the widget when the component is visible. An important set of properties for any component relates to its position and size within the container in which the component is placed; and another relates to the component's visual appearance.

Methods

The methods of a component determine what messages can be sent to it. Many of these messages correspond to getter and setter methods for the component's properties.

For example, `JTextField` provides the methods `setText(aString)` and `getText()` to allow the text property to be set or read. It is these methods that allow text fields to be used for both output and input: the programmer can use `setText(aString)` to output information via the text field, and `getText()` to input a string entered in the text field by the user.

Events

Swing components can be set up to react to particular events. For example, you can arrange that when the user clicks on a button, a particular action is carried out.

An event is implemented as an instance of a class defined in the package `java.awt` – the Abstract Windowing Toolkit (AWT) classes. To make a component react to an event, you need to write a method (called a handler) for the event that you are interested in (a mouse-click on the component, say). This handler has to be written in a special kind of class that implements a whole category of events that involve the component. Then you add a *listener* (an instance of the class in which this handler is implemented) to the component.

As a result, whenever the user clicks on the widget the listener picks this up and invokes the handler method you have written for that event. You can think of the listener as a process that is running in the background waiting for the user to do something to the widget. Of course, if the user does something that you have not written a handler for then nothing will happen. Note that each handler belongs to a listener, and the listener is added to a particular component. This means that different components can react to the same event in different ways, as each has its own handler for that event. It is also possible to add the same handler for a particular event to a number of components (i.e. handlers can be reused) if you want to provide alternative ways of producing the same result.

Component layout

The rules which influence how components are arranged on a frame or panel are specified by a layout manager. A layout manager is an object of a class defined in `java.awt` which implements the `LayoutManager` interface. Programmers can write their own custom layout managers, but Java already provides a wide range of different layout managers which suffice for most purposes. Although in M256 you are not required to do any programming involving layout managers, three kinds are used in the GUIs in this unit and these are briefly described below.

Layout managers

Border layout A `BorderLayout` splits a frame or panel into five regions as shown in Figure 26. The `North` and `South` regions always take up the full width of the container. The other dimensions of a region depend both on the space available and the size of the component the region contains. If a region is empty the other regions expand to take up the space available.

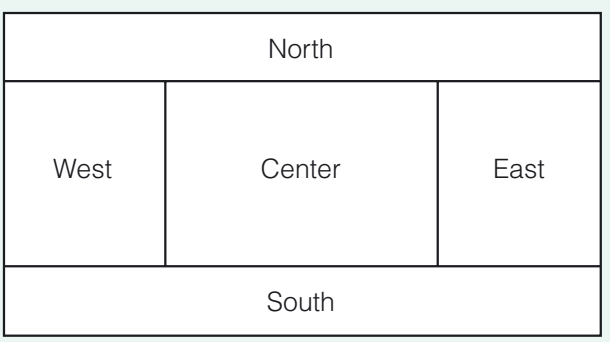


Figure 26 Border layout

Grid bag layout A `GridBagLayout` is like a table of cells. The rows and column widths can vary, adjacent cells can be merged, and so on – much like a spreadsheet, or a table inserted into a word-processed document.

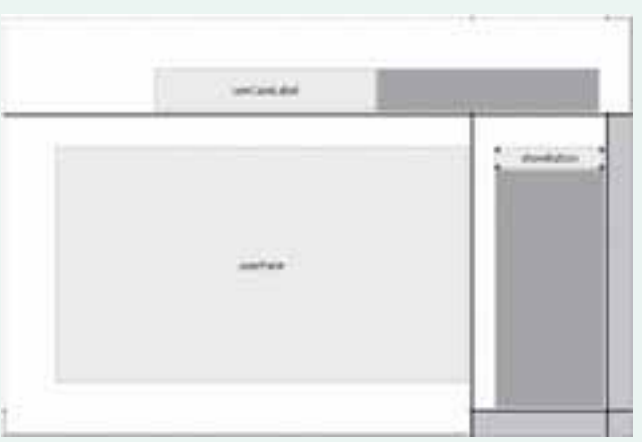


Figure 27 Grid bag layout

A `GridBagLayout` offers the highest level of control and flexibility of any layout manager. However, it is complicated to use, because so many parameters have to be set.

Null layout A `NullLayout` allows complete freedom of positioning and is very simple to work with. However, there are disadvantages: all the components have to be positioned manually, and the resulting design will not cope well if the window is resized; it may also not transfer reliably to other platforms. So this layout would not be used for a completed implementation, although it might be adequate for prototyping.

Glossary

Abstract Windowing Toolkit (AWT) Java classes, defined in the package `java.awt`, including classes implementing events and layout managers.

container A Swing component which contains other visual components. Every Swing GUI must have a top-level container in which the rest of the GUI components are nested.

event (in a GUI) Something that happens when a user interacts with a GUI; typically initiated by a mouse operation, although the keyboard also generates events. In Java an object is created to represent each event that occurs. Code must be associated with an event in order that the system can react appropriately to it.

event handler method A method in which event handling code is written.

event handling code Code that specifies the action to be taken in response to a particular event.

helper method A private method that exists simply to assist the work of another method in the same class.

layout manager A Java object that controls how Swing components can be laid out in a container.

listener An instance of the class in which a handler method is implemented.

persistence The ability of a system to save state from one execution to the next.

scroll pane A widget that displays scrollbars as and when they are required. If there are too many items, a vertical scroll bar will automatically appear. If an item is too wide, a horizontal scrollbar will automatically appear.

structured English A description of code which has some of the structure of code but without the syntax of a programming language.

Swing Java classes, defined in the package `java.swing`, used for programming GUIs.

top-level container A component which is the overall container for the components in a GUI.

visual component A Java implementation of a widget.

Index